

bok25

**基
础**

班、计算机组成原理



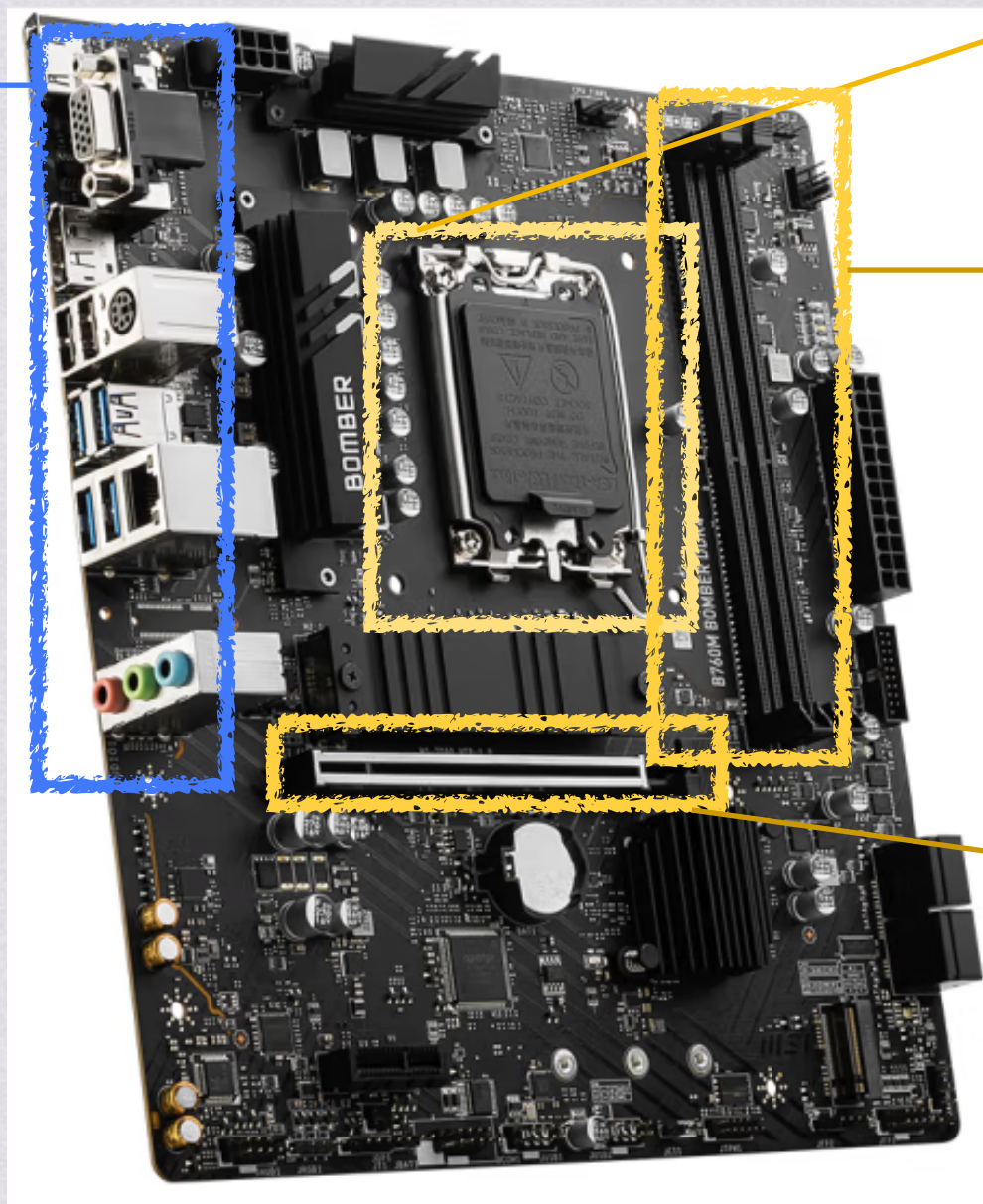
输入输出系统



输入输出系统

IO接口：连接外设和主机的一个“桥梁”

用于连接各种外设的接口
外设侧的通常称为外部IO接口



放置CPU的位置

连接内存位置

主机侧的称为
内部接口

连接显卡位置



输入输出系统

主机侧的称为内部接口：通过IO总线和内存、CPU相连

外设侧的通常称为外部IO接口：通过IO接口电缆（USB线、网线）连接到外设上

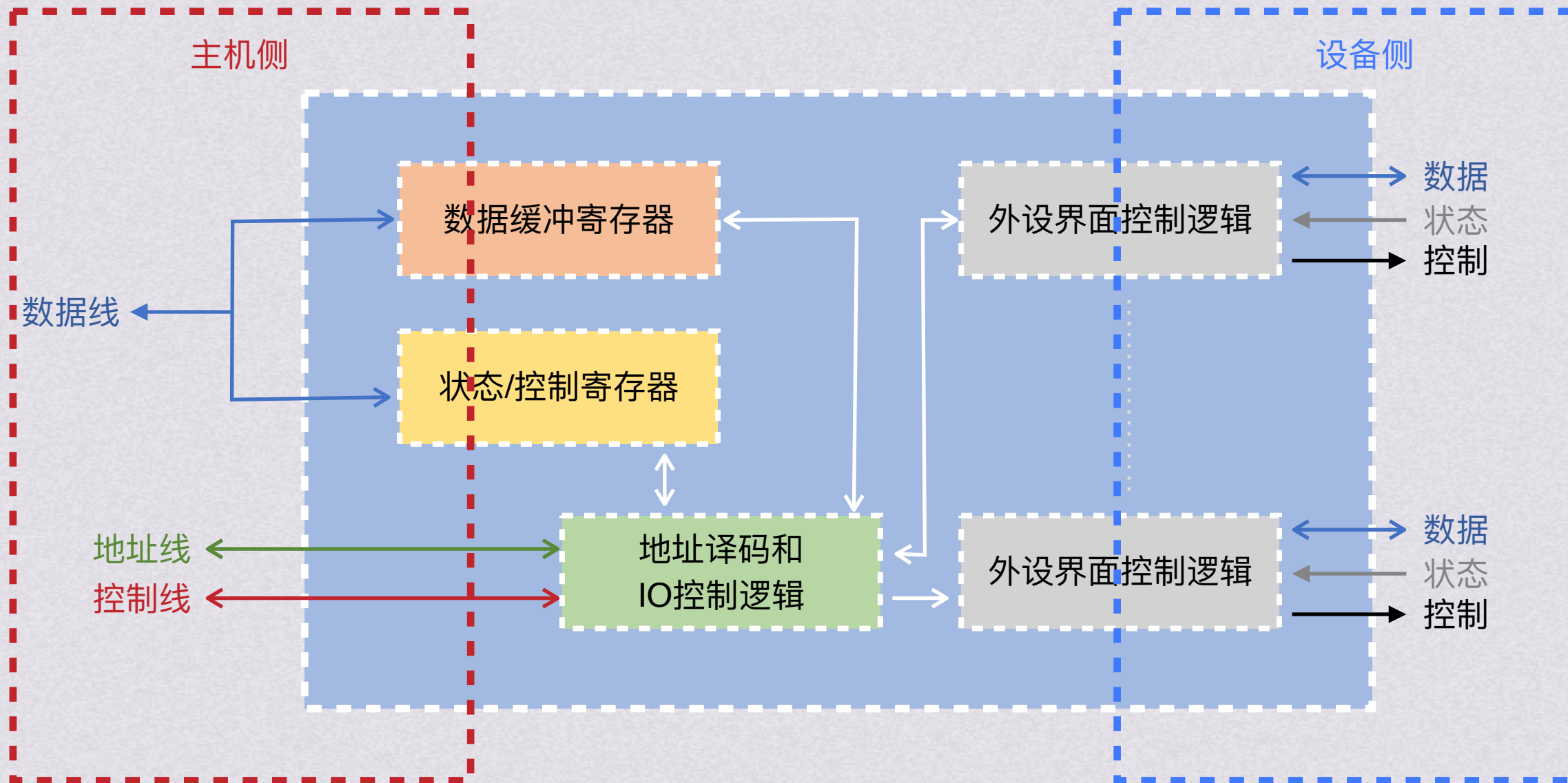


IO基本职能

- 1.数据缓冲：主存和CPU寄存器的存取速度非常快，外设速度较低，所以在IO接口中引入数据缓冲寄存器，匹配主机、外设的工作速度
- 2.错误或状态检测：在IO接口中提供状态寄存器保存各种状态信息
- 3.控制和定时：提供控制、定时逻辑，以接受系统总线来的控制命令和定时信号
- 4.数据格式转换：提供数据格式转换部件，使通过外部接口得到的数据转换为内部接口需要的格式或者反着转换



IO接口的通用结构



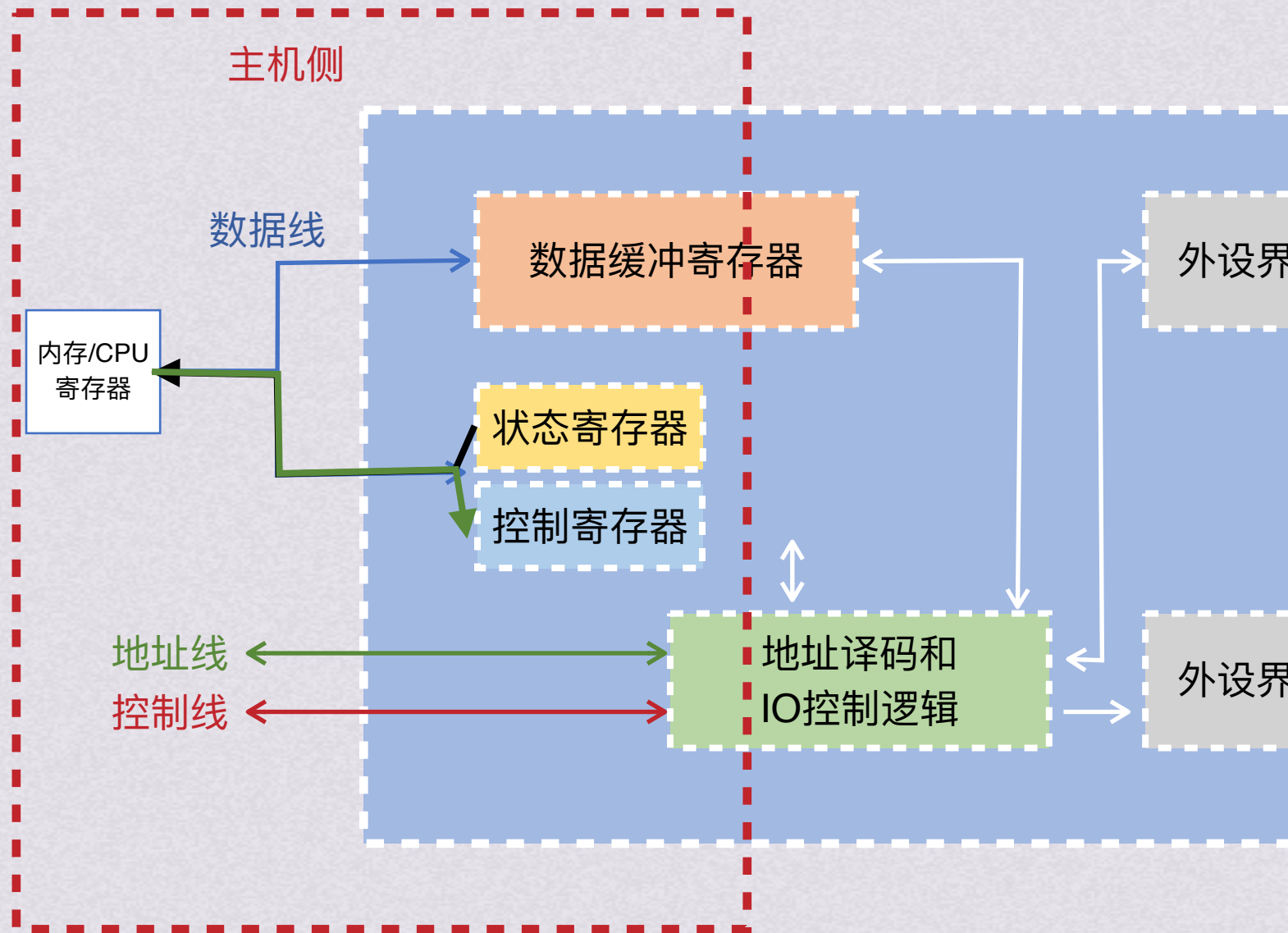


IO接口的通用结构

IO接口在主机侧通过IO总线与内存、CPU相连
通过数据线，在数据缓冲寄存器与内存或CPU
的寄存器之间进行数据传送
同时接口和设备状态信息被记录在状态寄存器
中

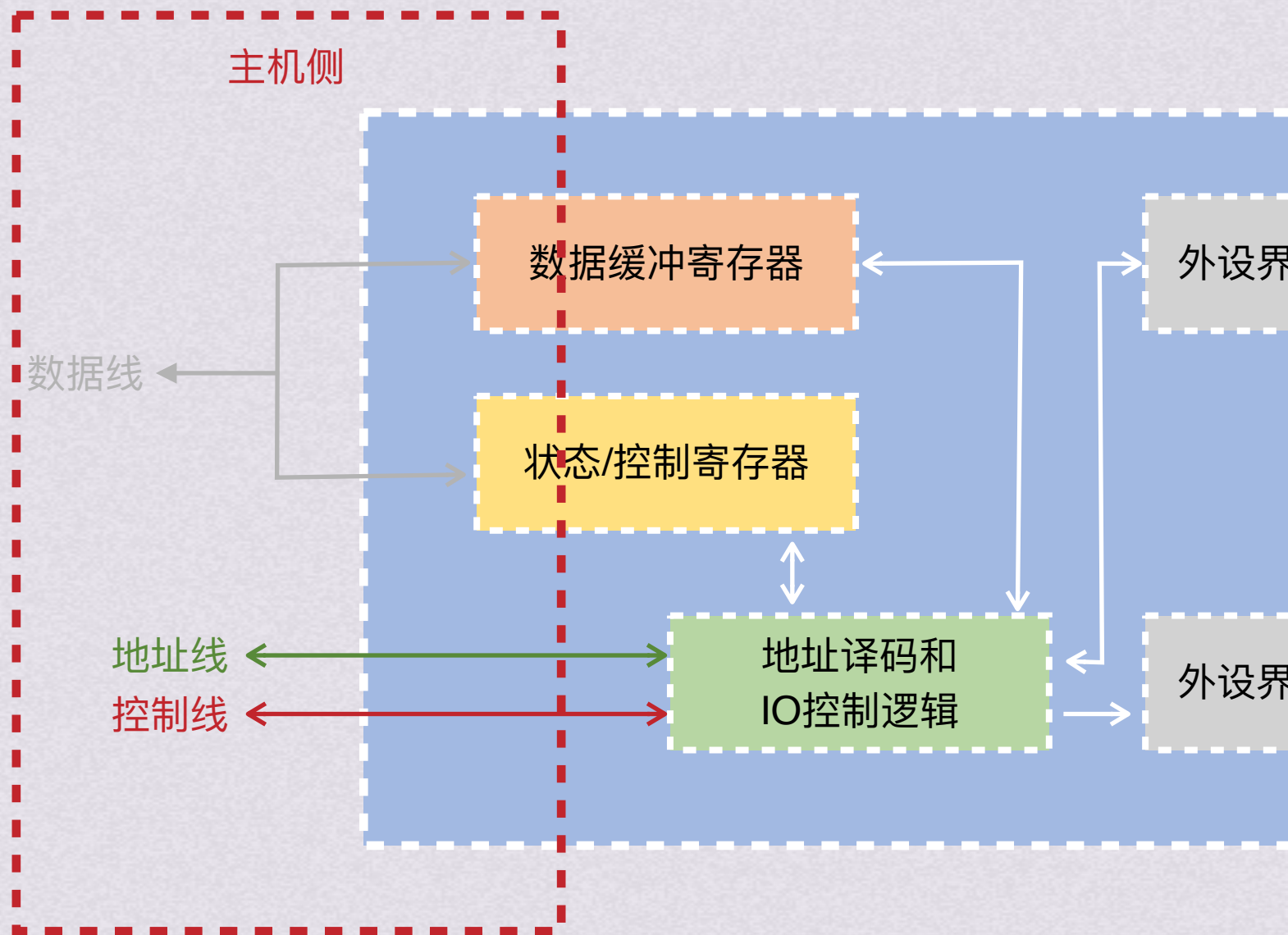
通过数据线将状态信息送到CPU，以供查用
CPU对外设的控制命令也通过数据线传送，
一般将其送到IO接口的控制寄存器

因为状态寄存器和控制寄存器传送方向相
反，CPU对他们的访问时间一般都是错开
的，所以有的IO接口将它们合而为一





IO接口的通用结构



地址线用于给出要访问的IO接口中寄存器的地址，它和读/写控制信号一起被送到IO接口中的控制逻辑部件中



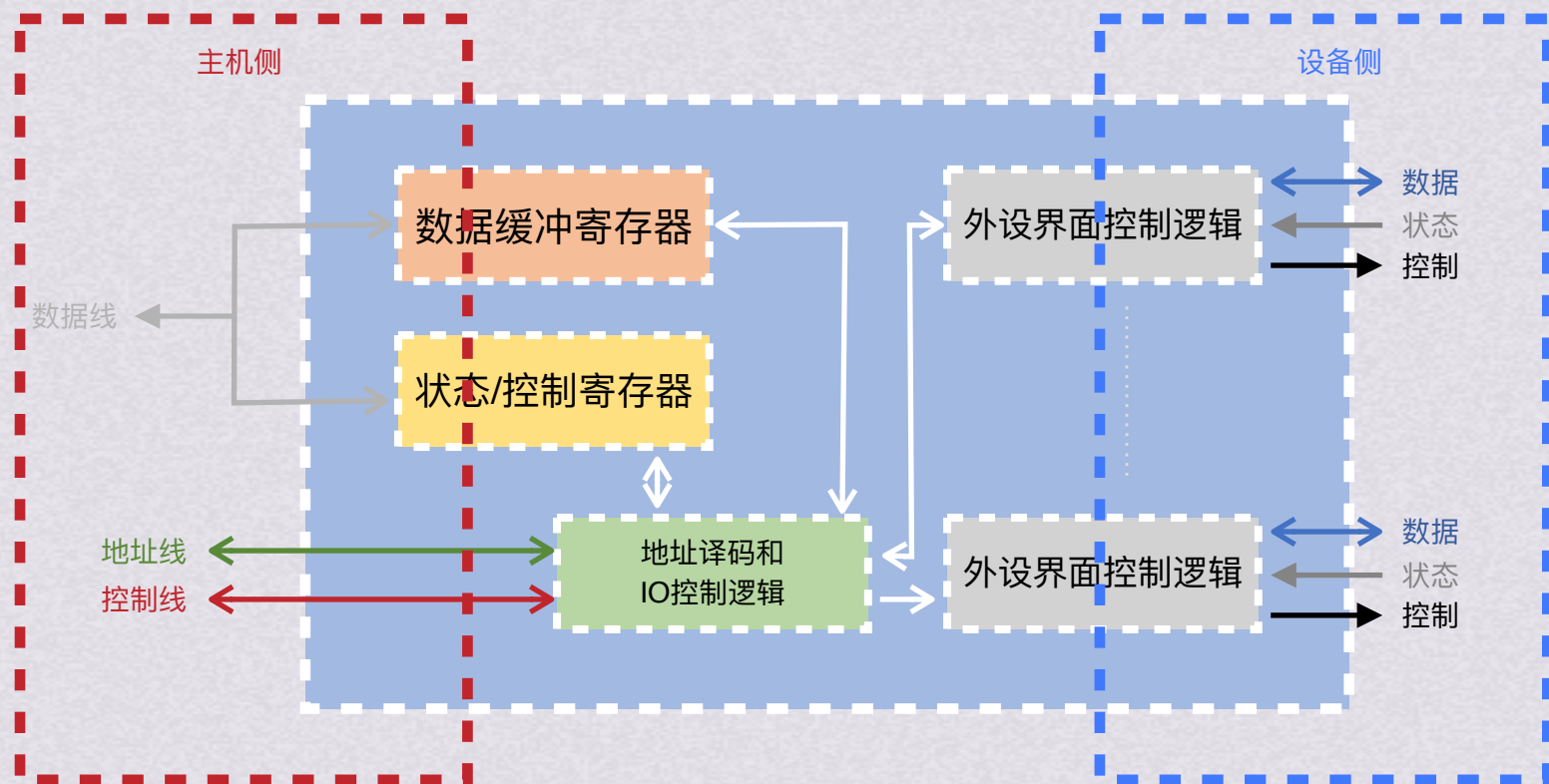
IO接口的通用结构

地址线用于给出要访问的IO接口中寄存器的地址，它和读/写控制信号一起被送到IO接口中的控制逻辑部件中

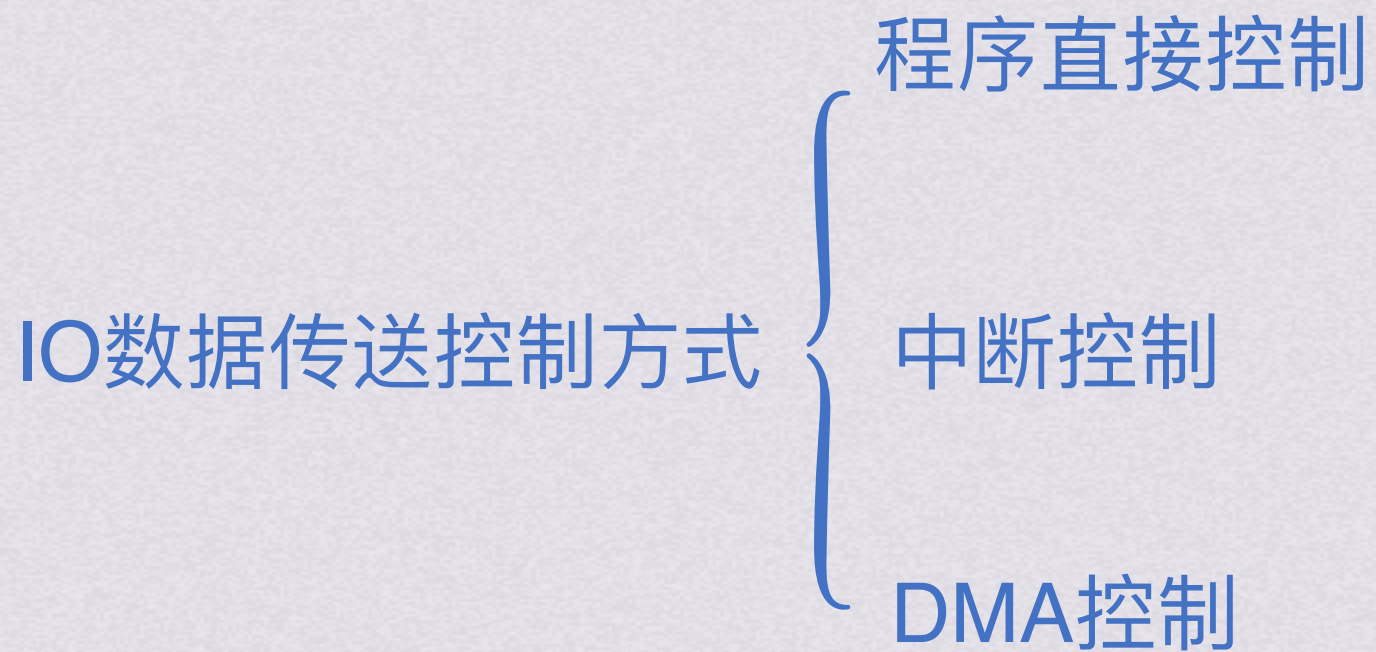
其中地址信息用以选择和主机交换数据的寄存器，通过控制线传来的读/写控制信号也有可能参与地址译码

控制线中还有一些仲裁信号和握手信号等也可被IO接口使用，IO接口中的IO控制逻辑还要能对控制寄存器中的命令字进行译码，

将得到的控制信号通过外设界面控制逻辑送外设，同时将数据缓冲寄存器的数据发送到外设或从外设接收数据到数据缓冲寄存器



还要具有收集外设状态到状态寄存器的功能





IO数据传送控制方式

程序直接控制 (程序查询方式)

通过程序来定时或持续查询接口状态

优点：简单易控制，外围接口控制逻辑少

缺点：CPU需要从外设接口读取状态，并在外设未就绪时一直处于忙等待
由于CPU速度比外设快得多，CPU在等待外设完成任务过程中浪费很多时间



IO数据传送控制方式 程序直接控制(程序查询方式)

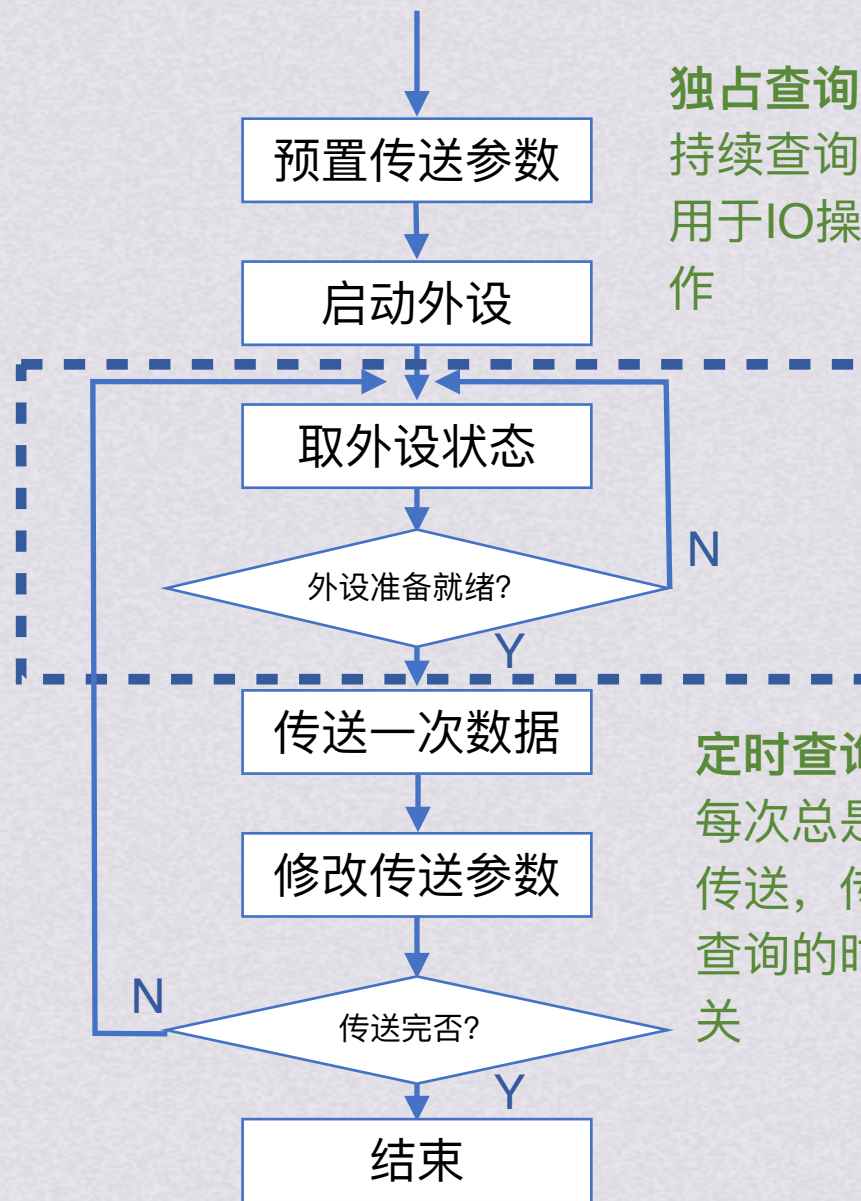
通过程序来定时或持续查询接口状态

优点：简单易控制，外围接口控制逻辑少

缺点：CPU需要从外设接口读取状态，并在外设未就绪时一直处于忙等待

由于CPU速度比外设快得多，CPU在等待外设完成任务过程中浪费很多时间

独占查询：一旦设备被启动，CPU就一直持续查询接口状态，CPU话费100%的时间用于IO操作，此时外设和CPU完全串行工作



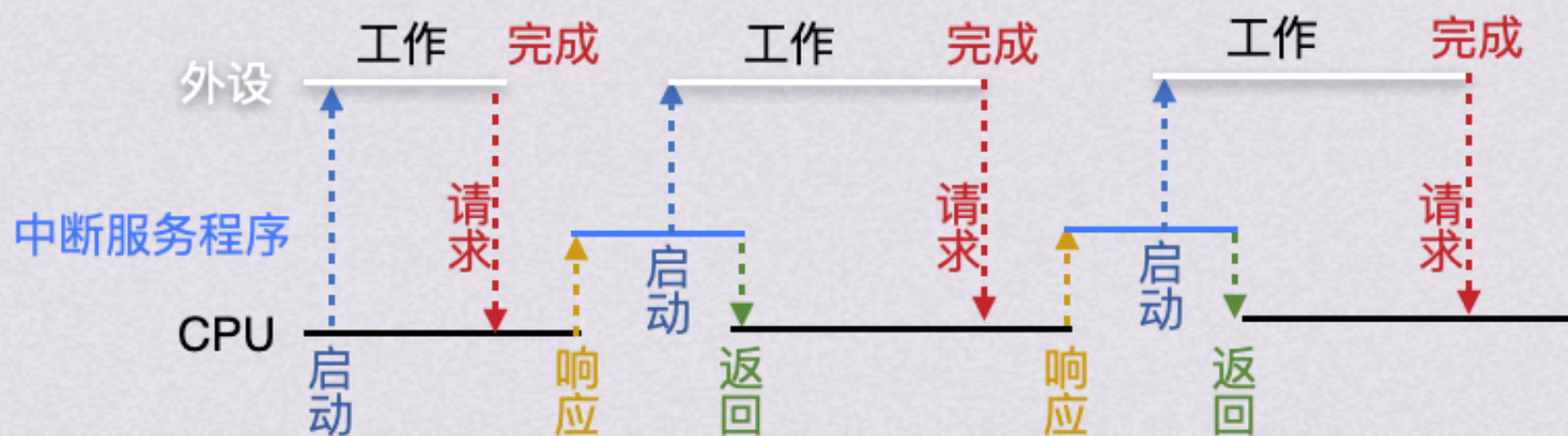
定时查询：CPU周期性地查询接口状态，每次总是等到条件满足才进行一个数据的传送，传送完成后返回到用户程序。定时查询的时间间隔与设备的数据传输速率有关



IO数据传送控制方式

中断控制 (程序中中断IO方式)

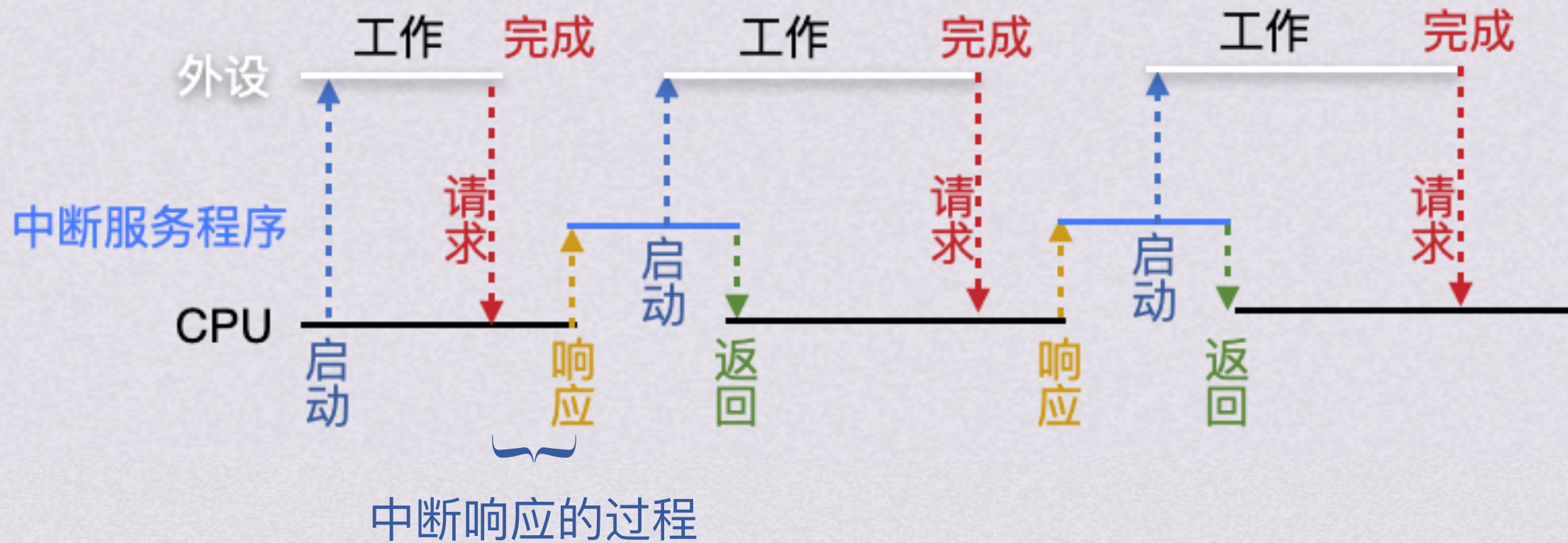
基本思想：外设和CPU并行工作，外设向CPU发出中断请求，在可以响应中断的条件下，CPU再暂时中止正在执行的程序，转去执行中断服务程序为外设服务





IO数据传输控制方式

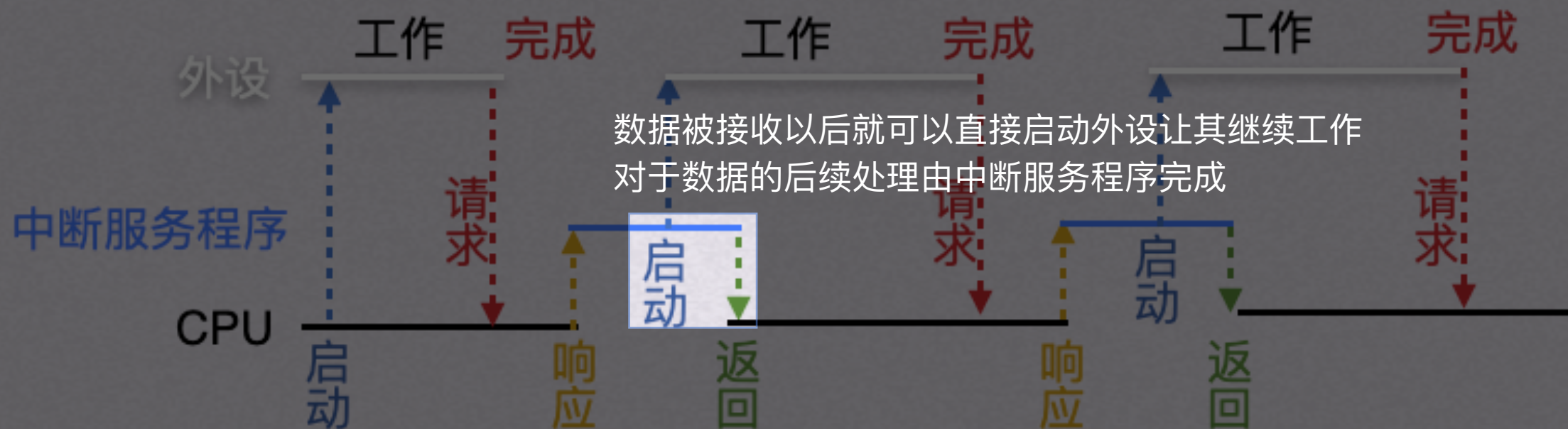
中断控制 (程序中中断IO方式)





IO数据传送控制方式

中断控制 (程序中中断IO方式)





IO数据传送控制方式

中断控制 (程序中中断IO方式)

中断响应的过程：

- 1.关中断
- 2.保存断点
- 3.引出中断服务程序

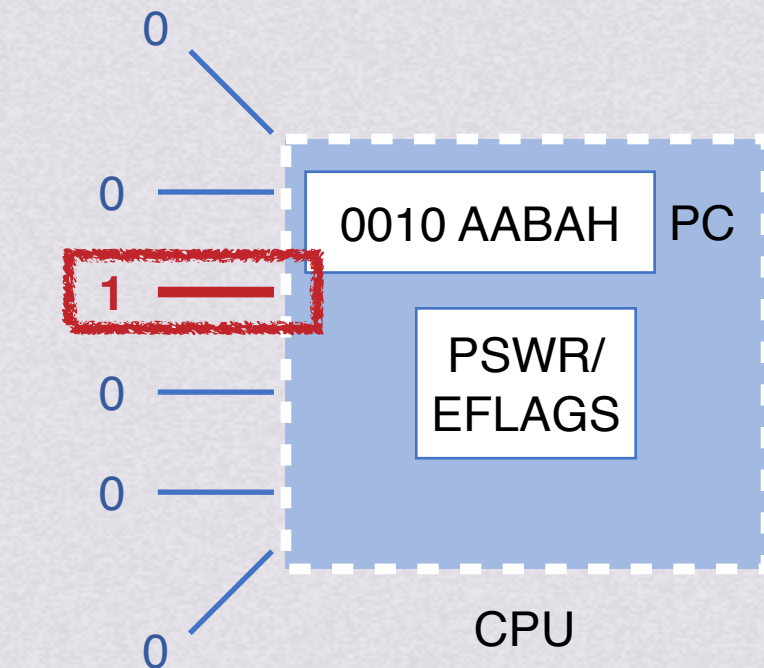


IO数据传送控制方式

中断控制(程序中中断IO方式) 中断响应的过程

1.关中断

在保护断点和现场的过程中，CPU不能响应更高级中断源的中断请求，否则可能导致断点或现场保存不完整，无法恢复现行程序



中断请求引脚

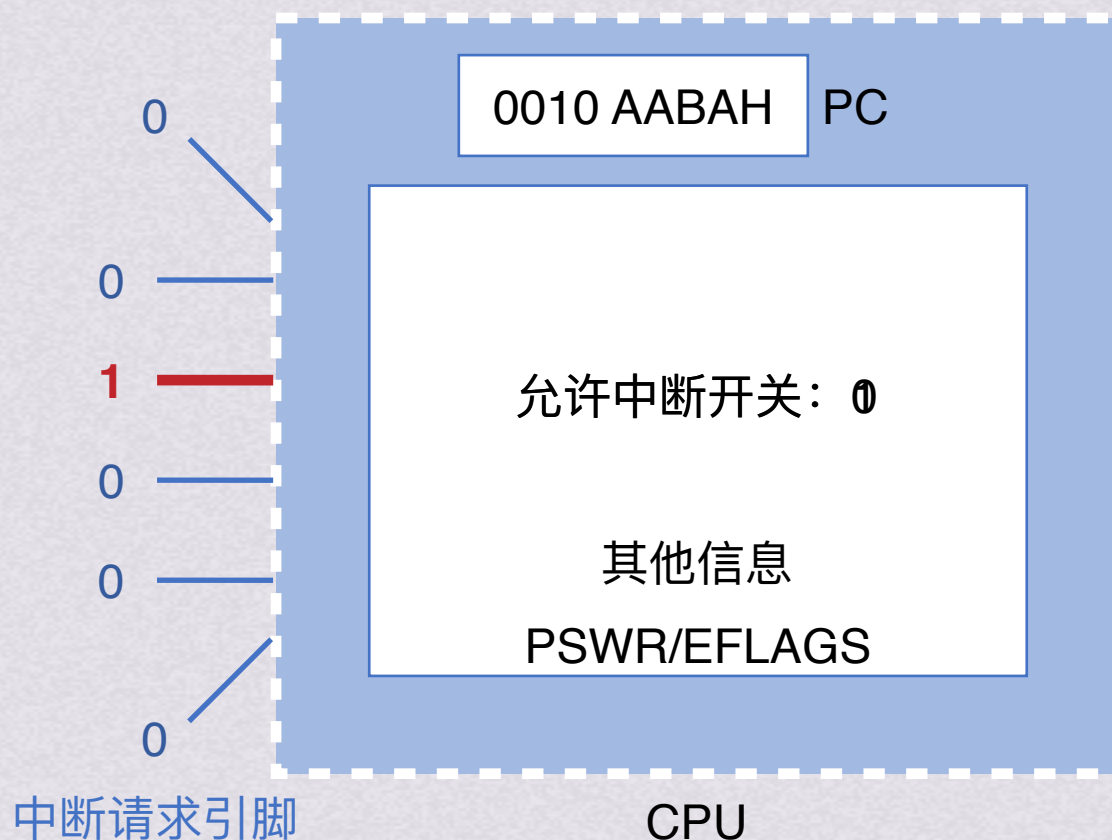


IO数据传送控制方式

中断控制(程序中中断IO方式) 中断响应的过程

1.关中断

在保护断点和现场的过程中，CPU不能响应更高级中断源的中断请求，否则可能导致断点或现场保存不完整，无法恢复现行程序



2.保存断点
3.引出中断服务程序
DMA控制



IO数据传送控制方式

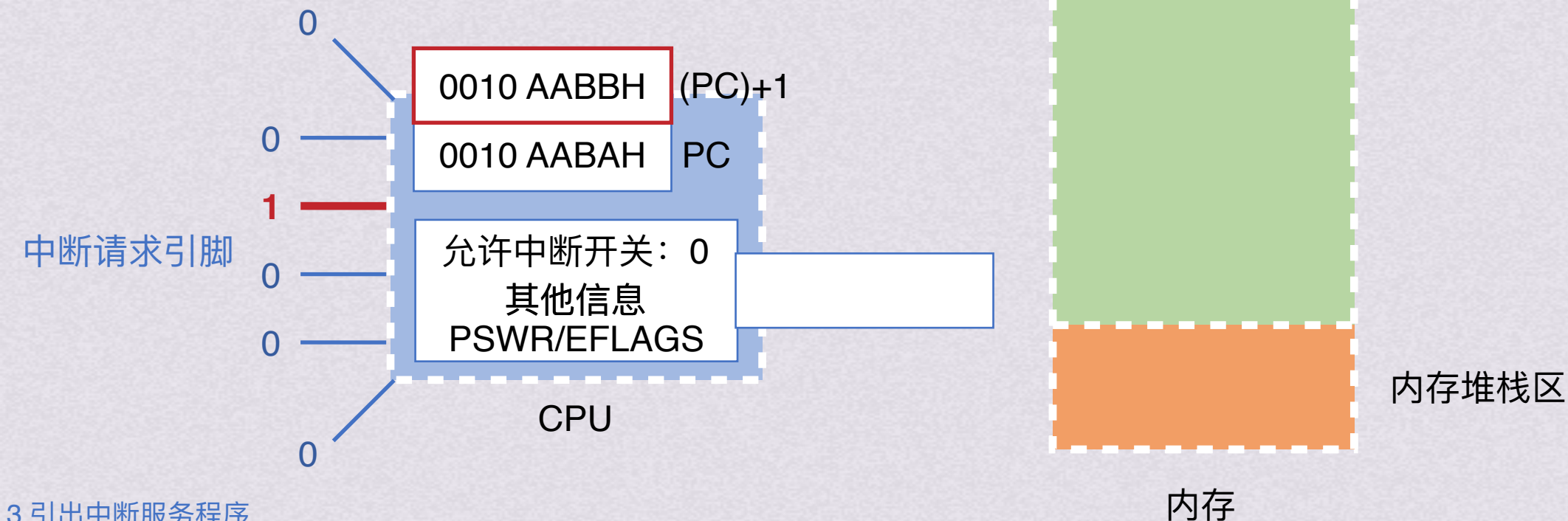
中断控制(程序中中断IO方式) 中断响应的过程

1.关中断

2.保存断点

将原程序的断点保存在栈或特定寄存器中

注意：异常的指令通常没有执行成功，异常处理后要重新执行，断点为当前指令地址；中断的断点是下一条指令的地址



3.引出中断服务程序
DMA控制



IO数据传送控制方式

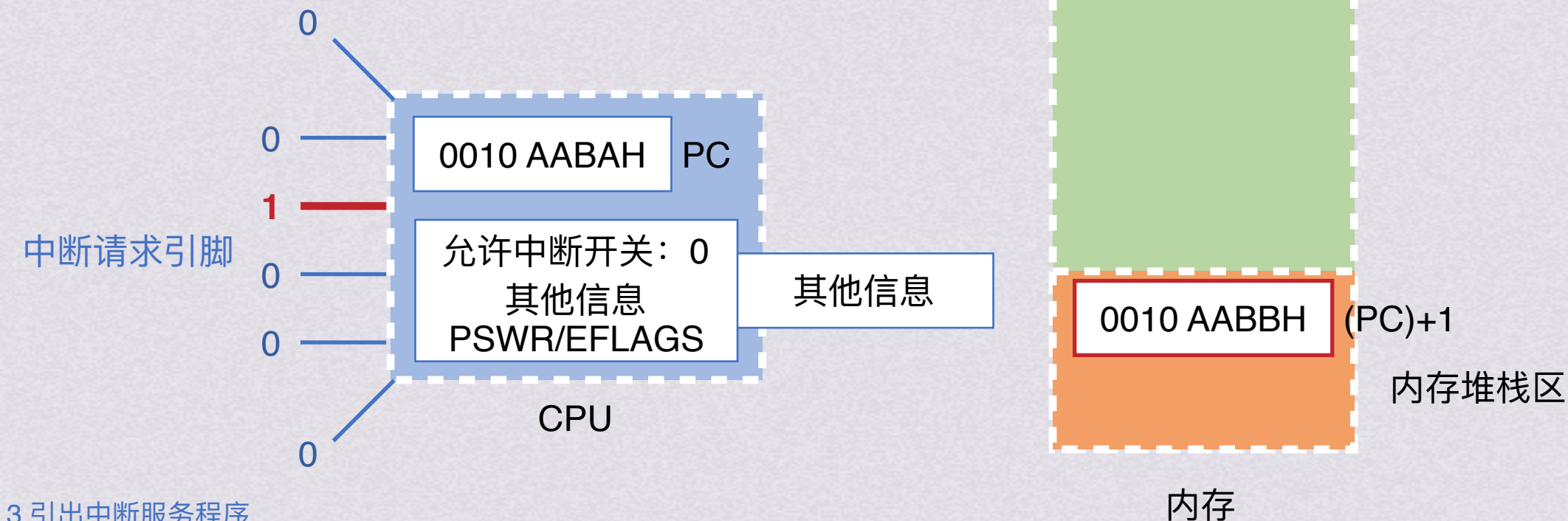
中断控制(程序中中断IO方式) 中断响应的过程

1.关中断

2.保存断点

将原程序的断点保存在栈或特定寄存器中

注意：异常的指令通常没有执行成功，异常处理后要重新执行，断点为当前指令地址；中断的断点是下一条指令的地址



3.引出中断服务程序
DMA控制



IO数据传送控制方式

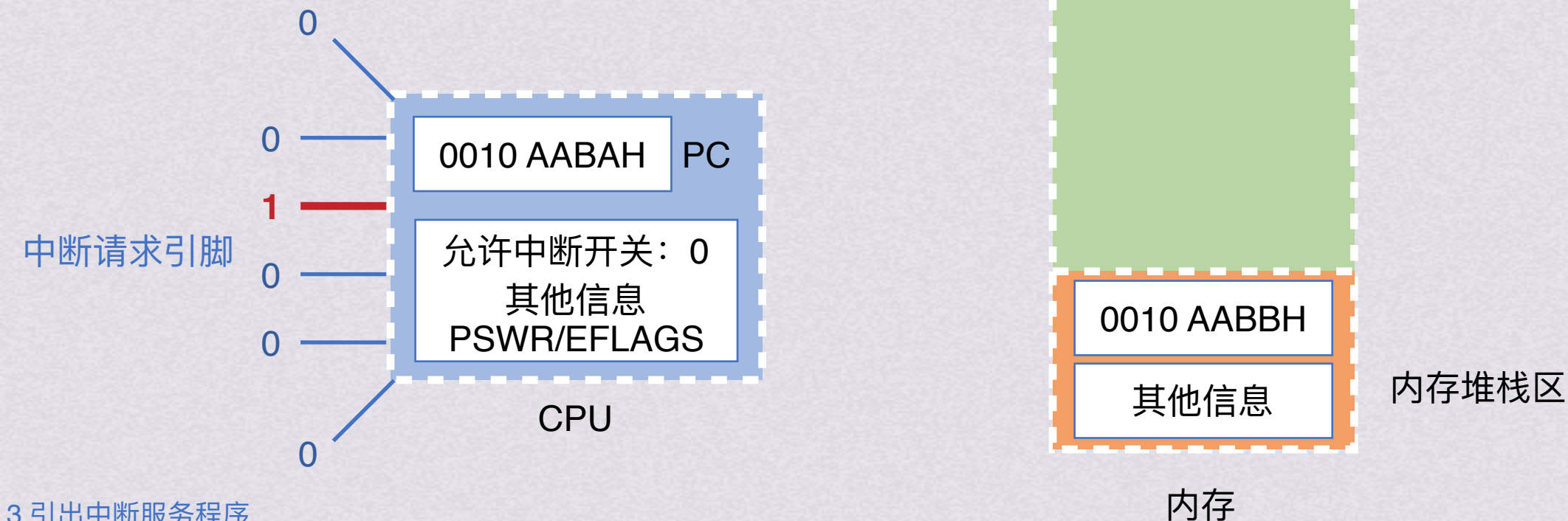
中断控制(程序中中断IO方式) 中断响应的过程

1.关中断

2.保存断点

将原程序的断点保存在栈或特定寄存器中

注意：异常的指令通常没有执行成功，异常处理后要重新执行，断点为当前指令地址；中断的断点是下一条指令的地址



3.引出中断服务程序
DMA控制



IO数据传送控制方式

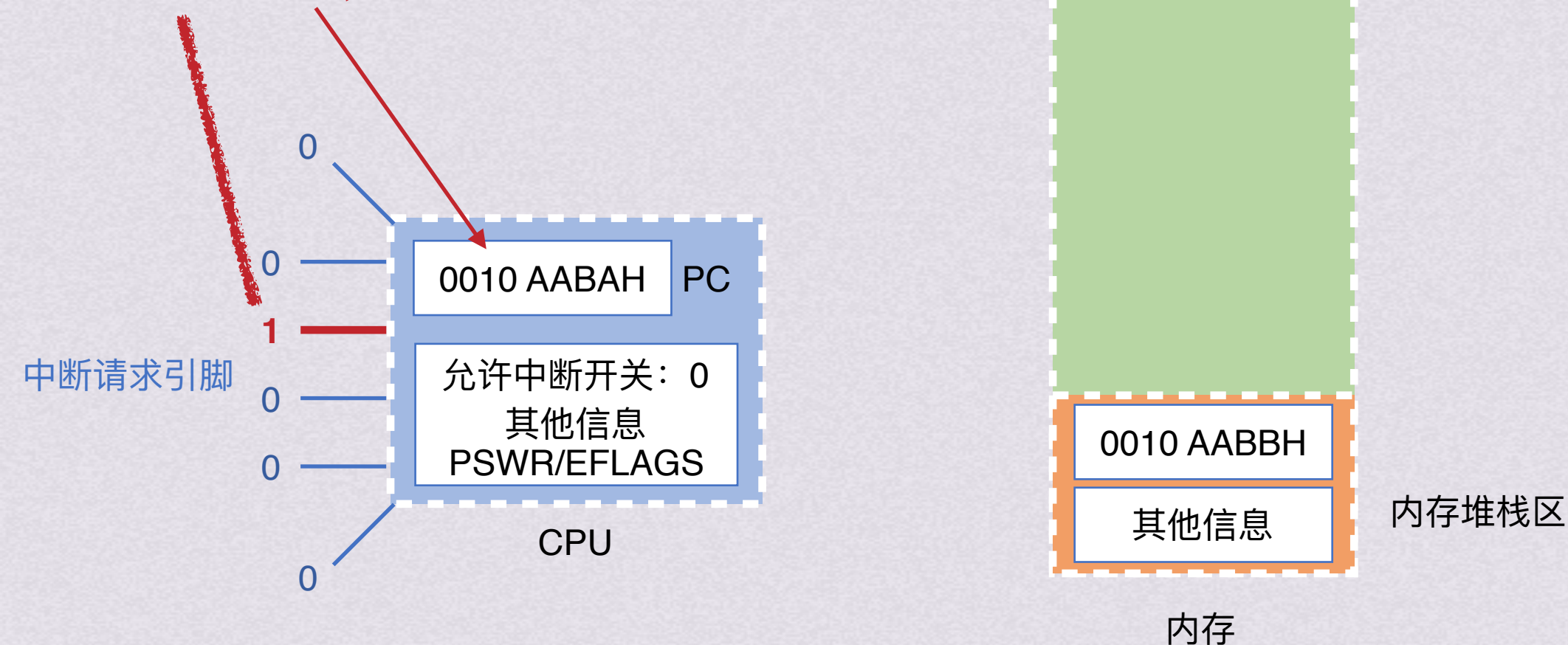
中断控制(程序中中断IO方式)

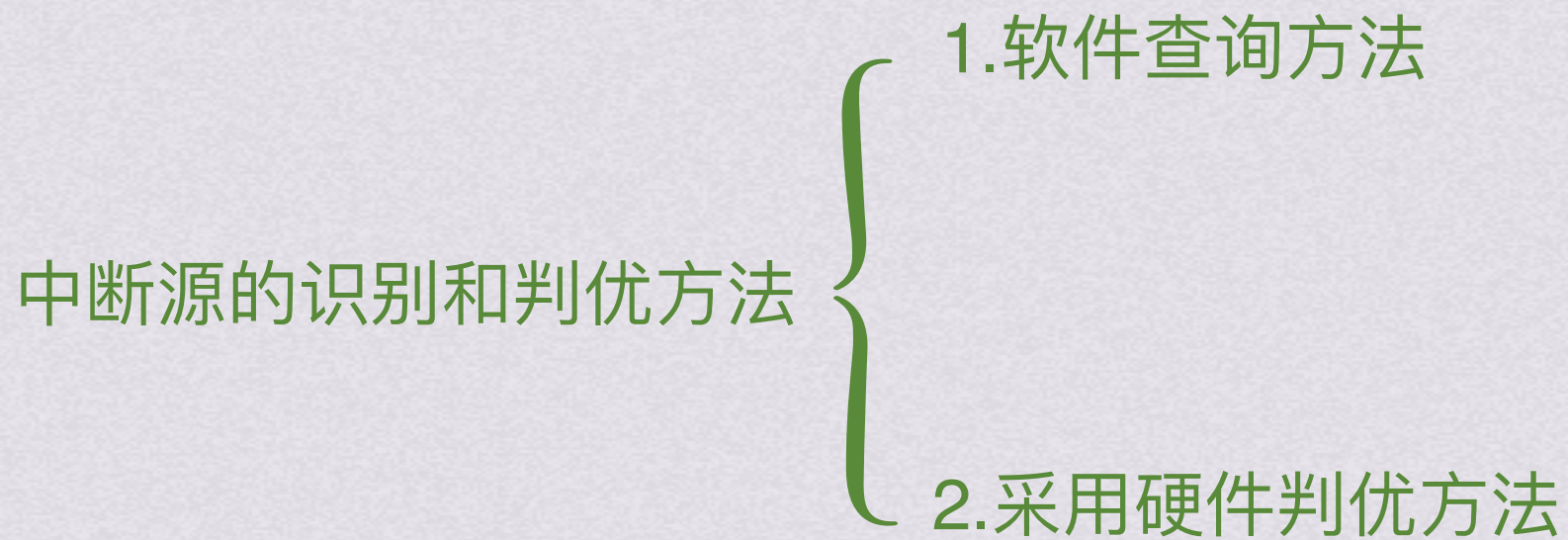
中断响应的过程

- 1.关中断
- 2.保存断点

3.引出中断服务程序

引脚对应中断服务程序入口地址（这个入口地址也叫中断向量）
假设为0275 88AFH，将其送入PC





中断源的识别和判优方法

1.软件查询方法

检测到中断请求时，通过中断响应，自动转到一个特定的中断查询程序，在中断查询程序中，按中断优先顺序依次查询哪个设备有中断请求，转到第一个查询到有请求的中断服务程序去执行



序号	是否中断请求	中断服务程序起始地址
1	0	8217 85B0H
2	0	8217 8600H
3	1	8217 8650H
4	0	8217 86A0H

软件进行中断识别方式，硬件结构很简单，且可软件更改查询顺序改变中断响应优先级
但因软件查询速度慢，中断请求可能无法得到实时响应

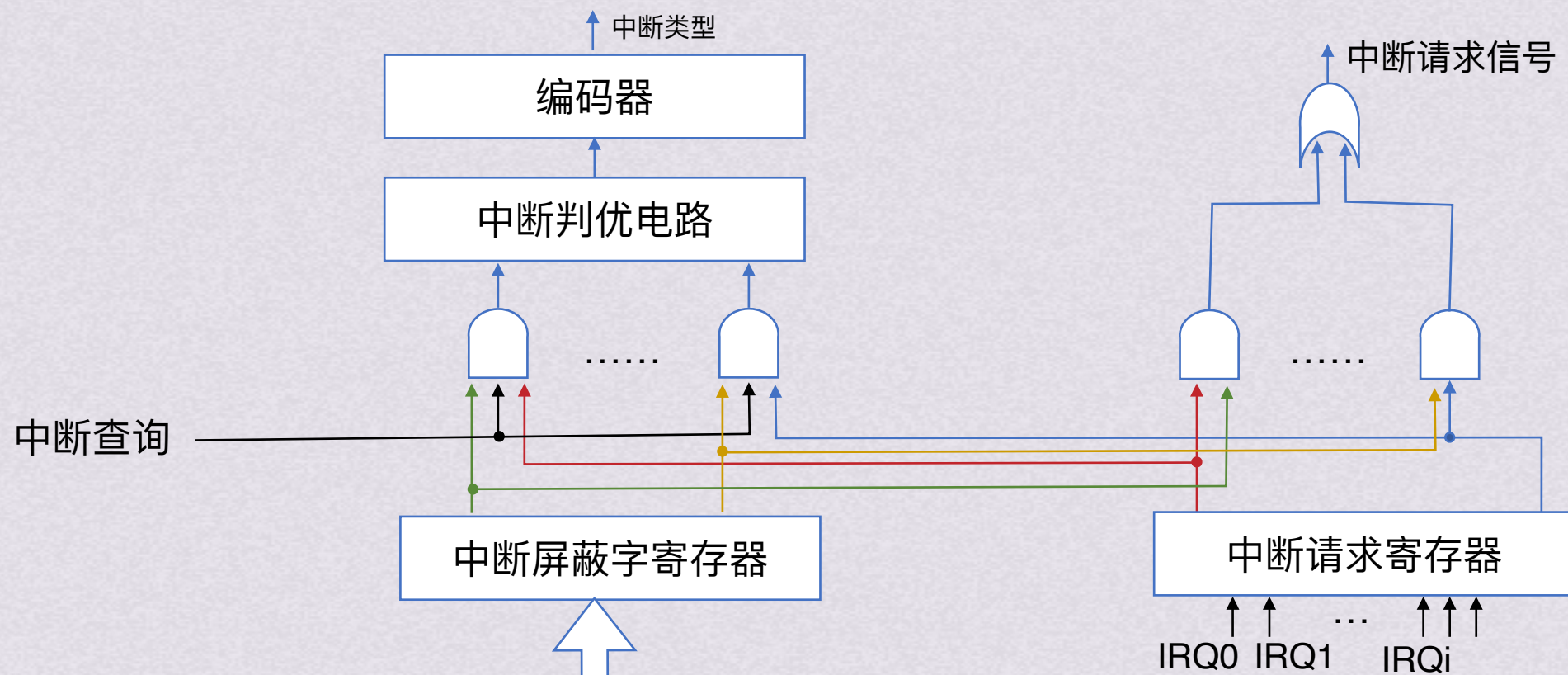
2.采用硬件判优方法 DMA控制



中断源的识别和判优方法

2.采用硬件判优方法

是一种向量中断方式，根据中断控制器接口中的中断判优电路和编码器等，得到当前所有未被屏蔽的中断请求中具有最高响应优先权的中断源类型（即中断源标识信息），通过编码器输出的中断源标识信息最终被送到CPU，CPU根据信息找到对应中断服务程序的首地址PC和初始PSW



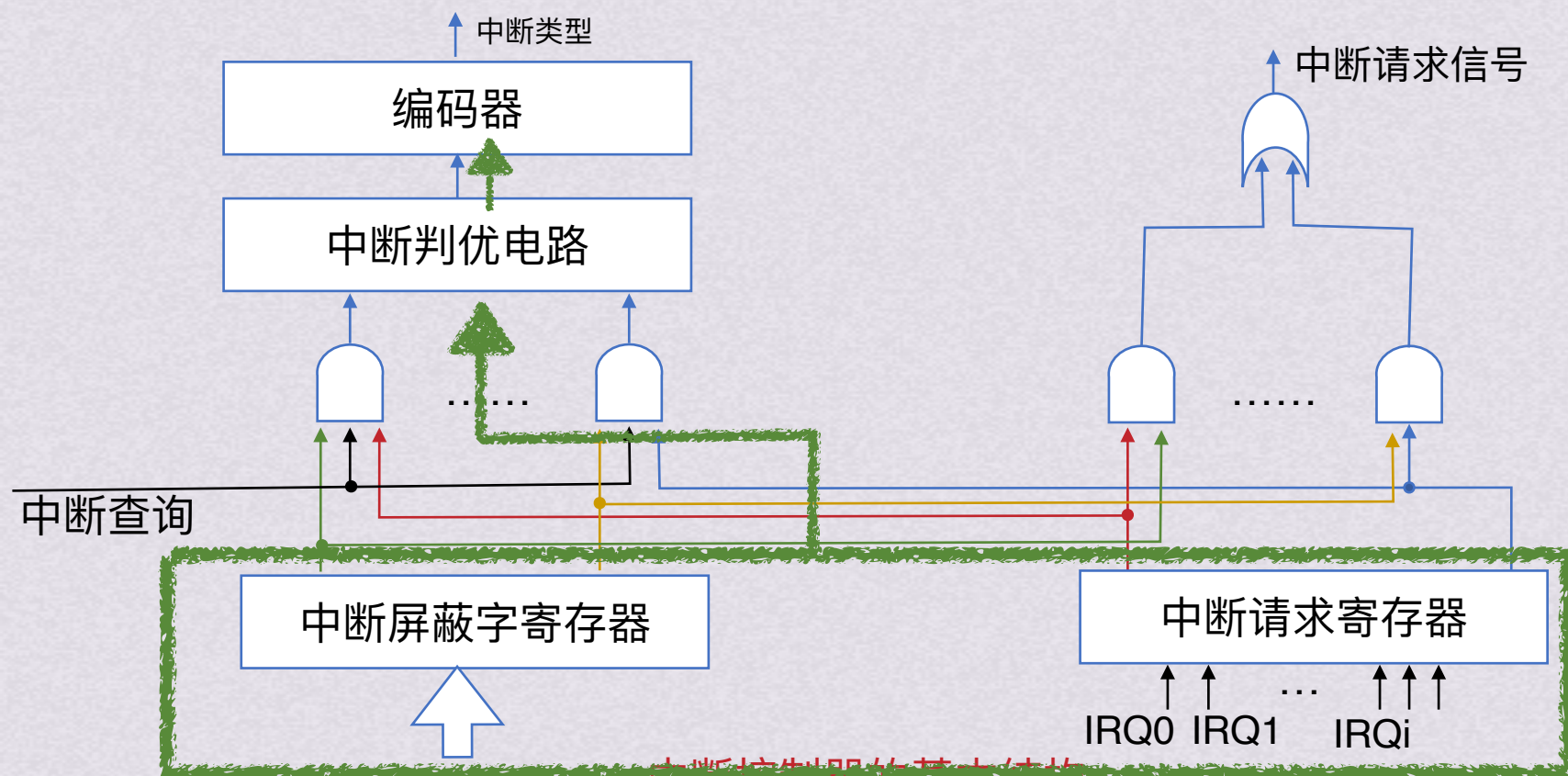
中断控制器的基本结构



中断源的识别和判优方法

2.采用硬件判优方法

是一种向量中断方式，根据中断控制器接口中的中断判优电路和编码器等，得到当前所有未被屏蔽的中断请求中具有最高响应优先权的中断源类型（即中断源标识信息），通过编码器输出的中断源标识信息最终被送到CPU，CPU根据信息找到对应中断服务程序的首地址PC和初始PSW



中断控制器的基本结构

现代计算机大多采用独立请求方式进行中断源的识别、判优。

各个中断请求信号和对应中断屏蔽位进行“与”操作后，送到优先级分析器中进行排队判优

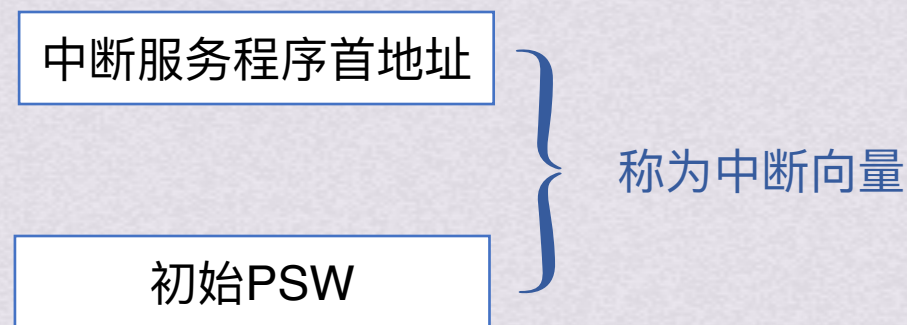
最后通过编码器输出对应中断类型号，即中断源标识信息



中断源的识别和判优方法

2.采用硬件判优方法

是一种向量中断方式，根据中断控制器接口中的中断判优电路和编码器等，得到当前所有未被屏蔽的中断请求中具有最高响应优先权的中断源类型（即中断源标识信息），通过编码器输出的中断源标识信息最终被送到CPU，CPU根据信息找到对应中断服务程序的首地址PC和初始PSW





中断源的识别和判优方法

2.采用硬件判优方法

1.软件查询方法

是一种向量中断方式，根据中断控制器接口中的中断判优电路和编码器等，得到当前所有未被屏蔽的中断请求中具有最高响应优先权的中断源类型（即中断源标识信息），通过编码器输出的中断源标识信息最终被送到CPU，CPU根据信息找到对应中断服务程序的首地址PC和初始PSW

中断服务程序首地址	中断向量
初始PSW	
中断服务程序首地址	中断向量
初始PSW	
中断服务程序首地址	中断向量
初始PSW	
中断服务程序首地址	中断向量
初始PSW	

.....



中断源的识别和判优方法

2.采用硬件判优方法

是一种向量中断方式，根据中断控制器接口中的中断判优电路和编码器等，得到当前所有未被屏蔽的中断请求中具有最高响应优先权的中断源类型（即中断源标识信息），通过编码器输出的中断源标识信息最终被送到CPU，CPU根据信息找到对应中断服务程序的首地址PC和初始PSW



中断向量表

每个中断向量在中断向量表中的位置可以用对应表项的编号来得到



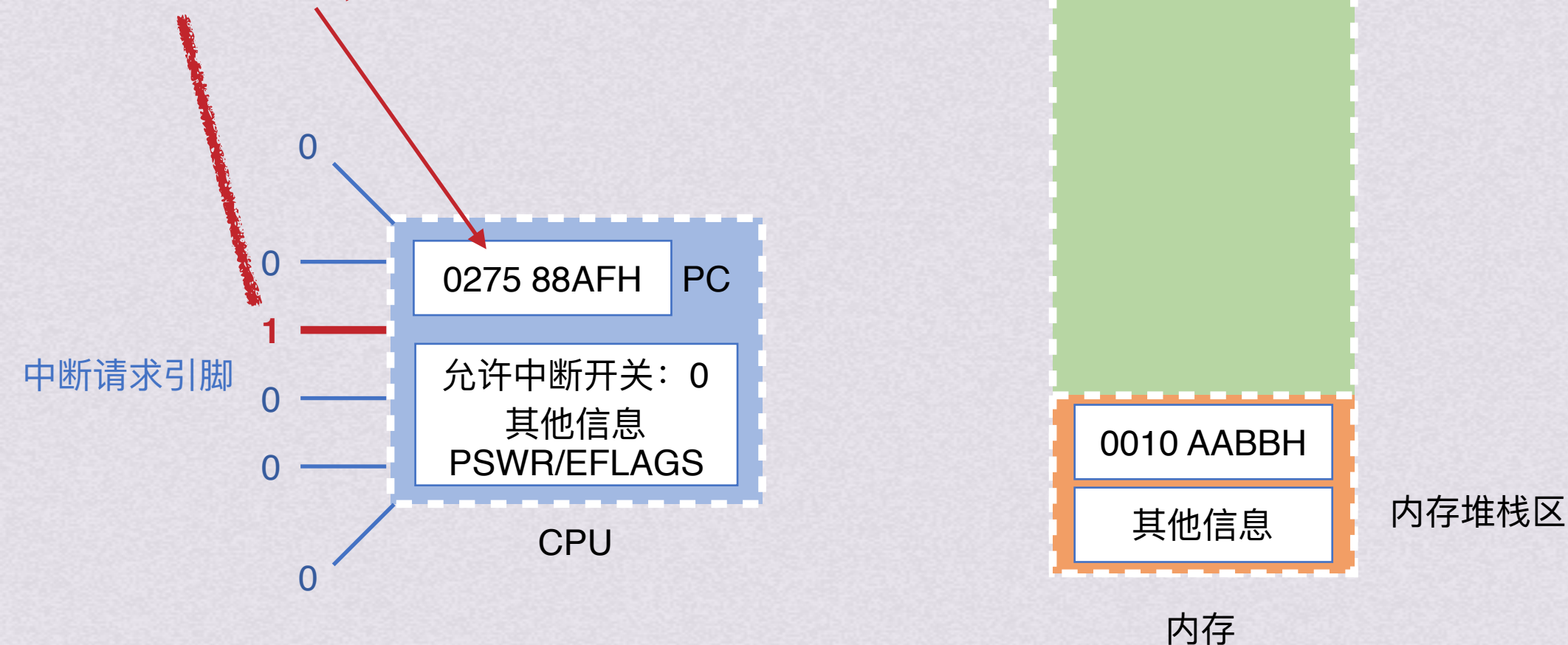
IO数据传送控制方式

中断控制(程序中中断IO方式) 中断响应的过程

- 1.关中断
- 2.保存断点

3.引出中断服务程序

引脚对应中断服务程序入口地址（这个入口地址也叫中断向量）
假设为0275 88AFH，将其送入PC





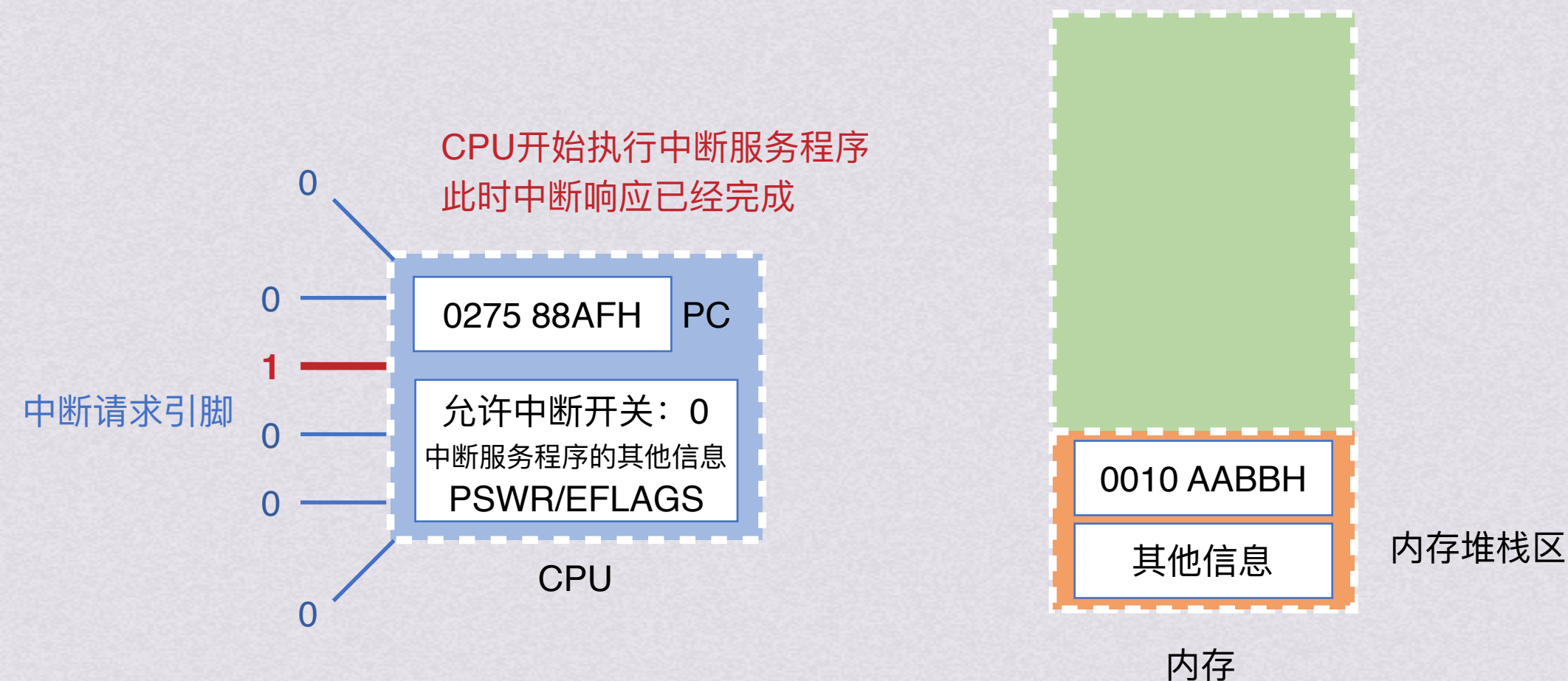
IO数据传送控制方式

中断控制(程序中中断IO方式)

中断响应的过程

- 1.关中断
- 2.保存断点

3.引出中断服务程序





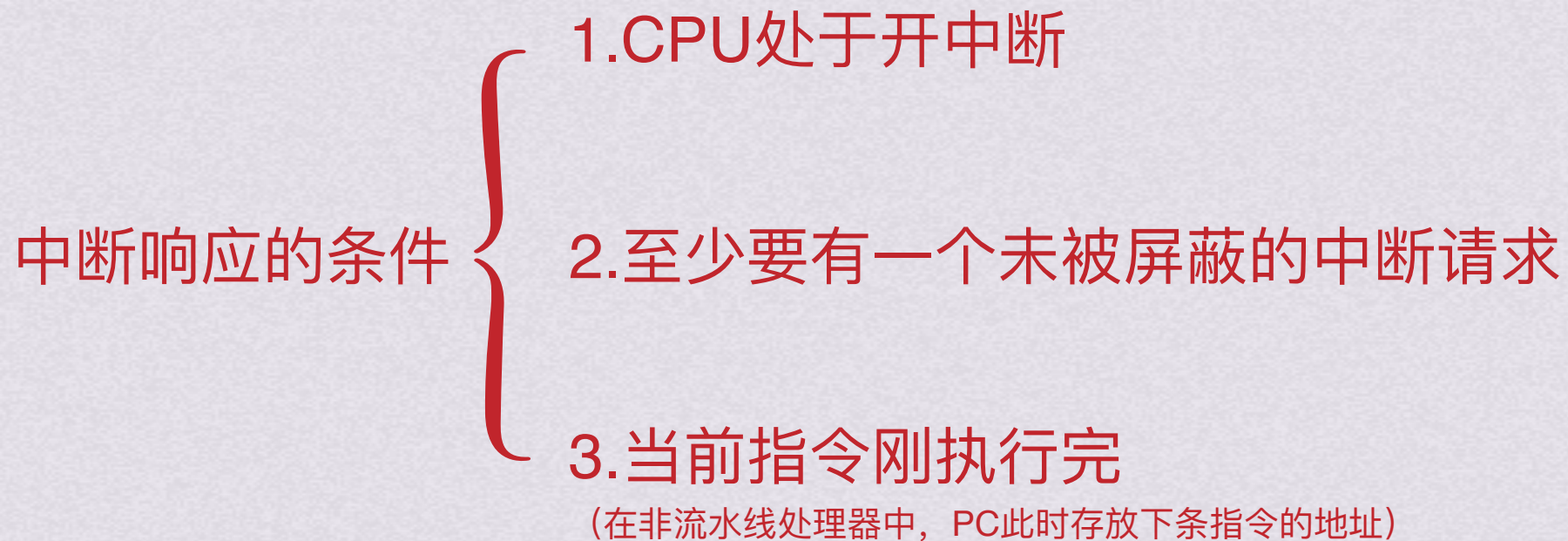
IO数据传送控制方式

中断控制(程序中断IO方式)

中断响应的过程
也称**中断隐指令**

- 1.关中断
- 2.保存断点
- 3.引出中断服务程序

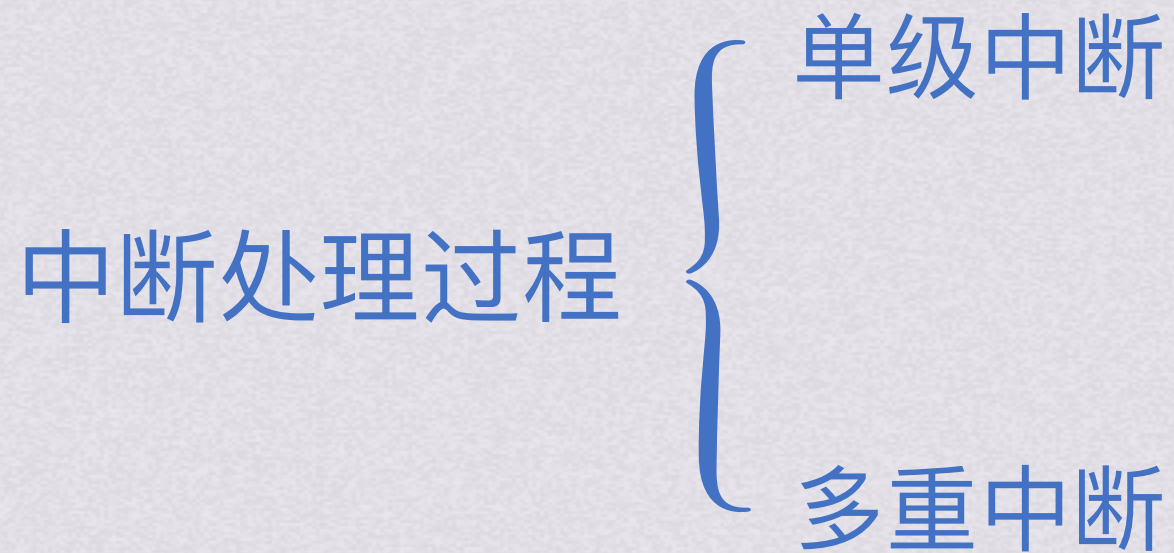
中断隐指令是硬件直接实现的一系列自动操作





IO数据传送控制方式

中断控制(程序中断IO方式)





IO数据传送控制方式

中断控制(程序中断IO方式)

中断处理过程

单级中断

中断响应
(中断隐指令)

关中断

保存断点

中断服务程序寻址

保存现场

执行中断服务程序

恢复现场

开中断

中断返回

中断处理



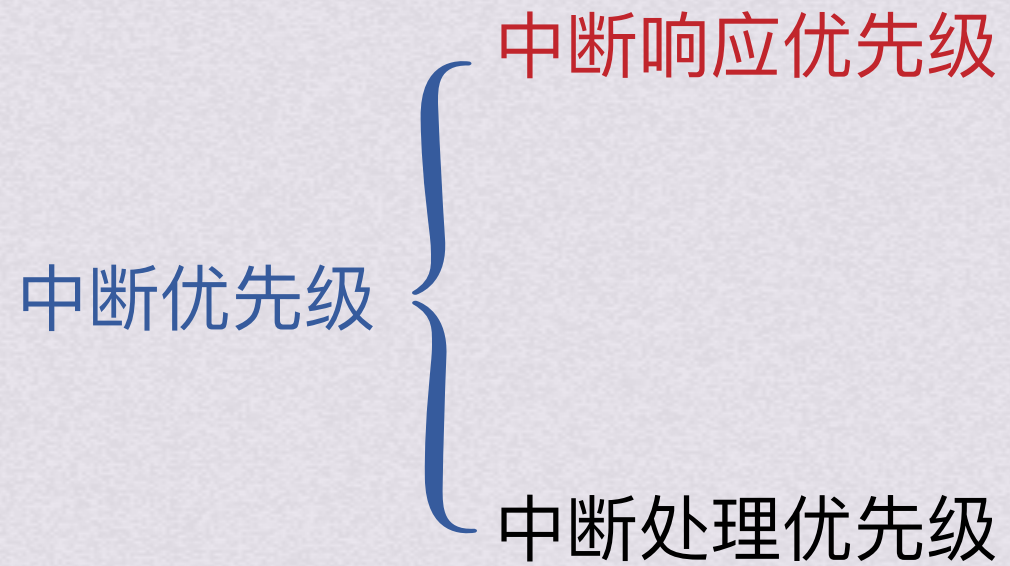
IO数据传送控制方式

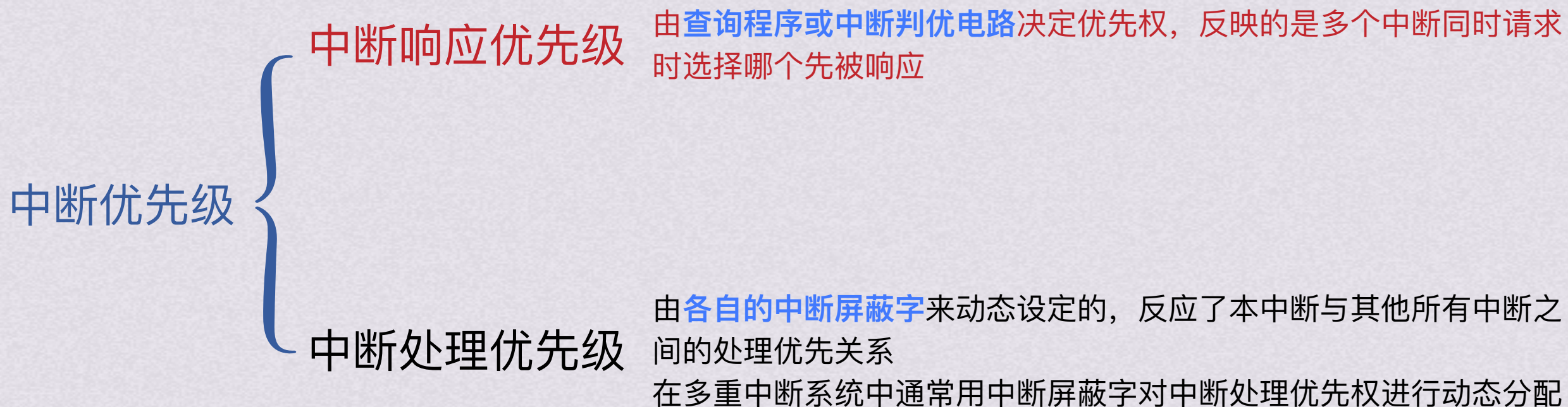
中断控制(程序中断IO方式)

中断处理过程

多重中断

之前课程中我们提到，有可能在处理一个中断的时候出现了第二个中断请求；
或者同时出现了多个中断请求
这种情况该如何处理呢？







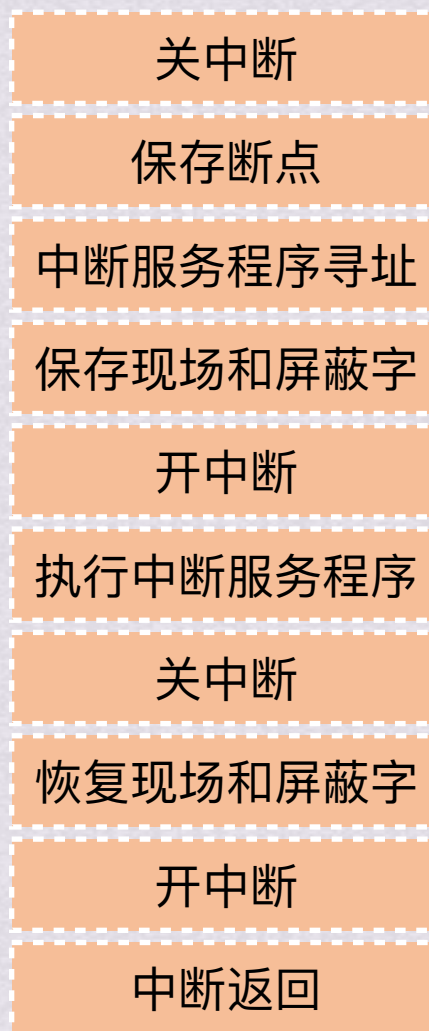
IO数据传送控制方式

中断控制(程序中中断IO方式)

中断处理过程 多重中断

中断响应
(中断隐指令)

中断处理



多重中断一般指可屏蔽中断



IO数据传送控制方式

中断控制(程序中中断IO方式)

中断处理过程 多重中断

中断响应
(中断隐指令)

中断处理

关中断

保存断点

中断服务程序寻址

保存现场和屏蔽字

开中断

执行中断服务程序

关中断

恢复现场和屏蔽字

开中断

中断返回

多重中断一般指可屏蔽中断

由于在中断处理过程中才会开中断，因此在隐指令的关中断到开中断之间都不会响应中断



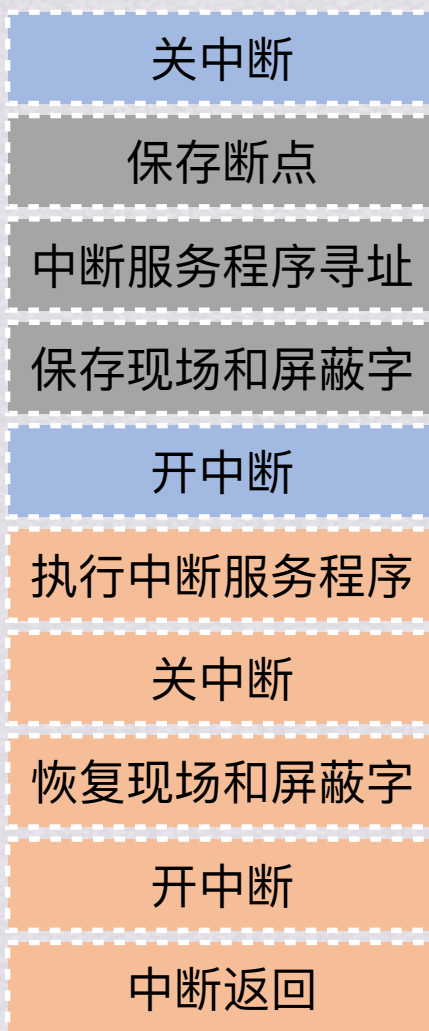
IO数据传送控制方式

中断控制(程序中中断IO方式)

中断处理过程 多重中断

中断响应
(中断隐指令)

中断处理



多重中断一般指可屏蔽中断

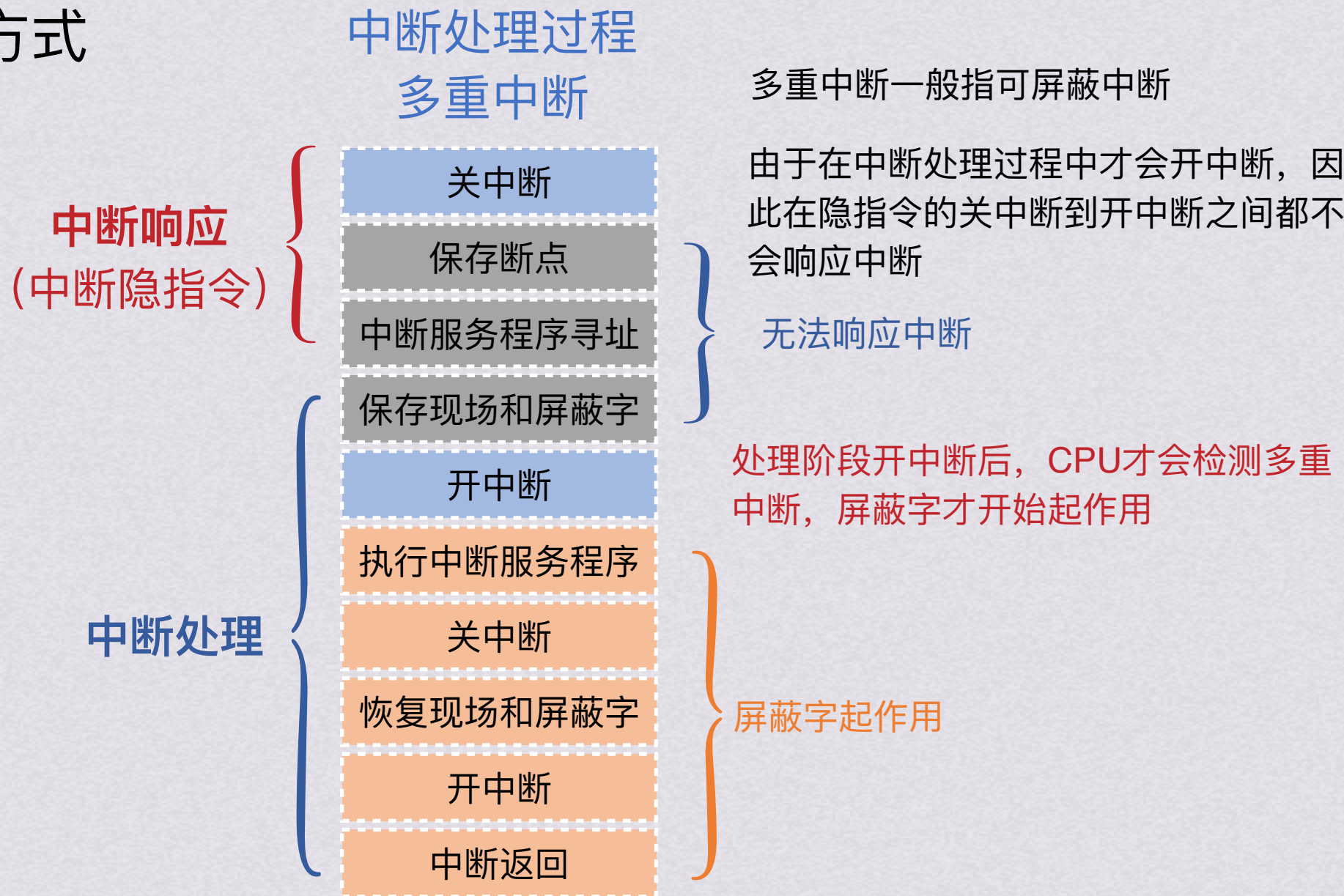
由于在中断处理过程中才会开中断，因此在隐指令的关中断到开中断之间都不会响应中断

无法响应中断



IO数据传送控制方式

中断控制(程序中中断IO方式)

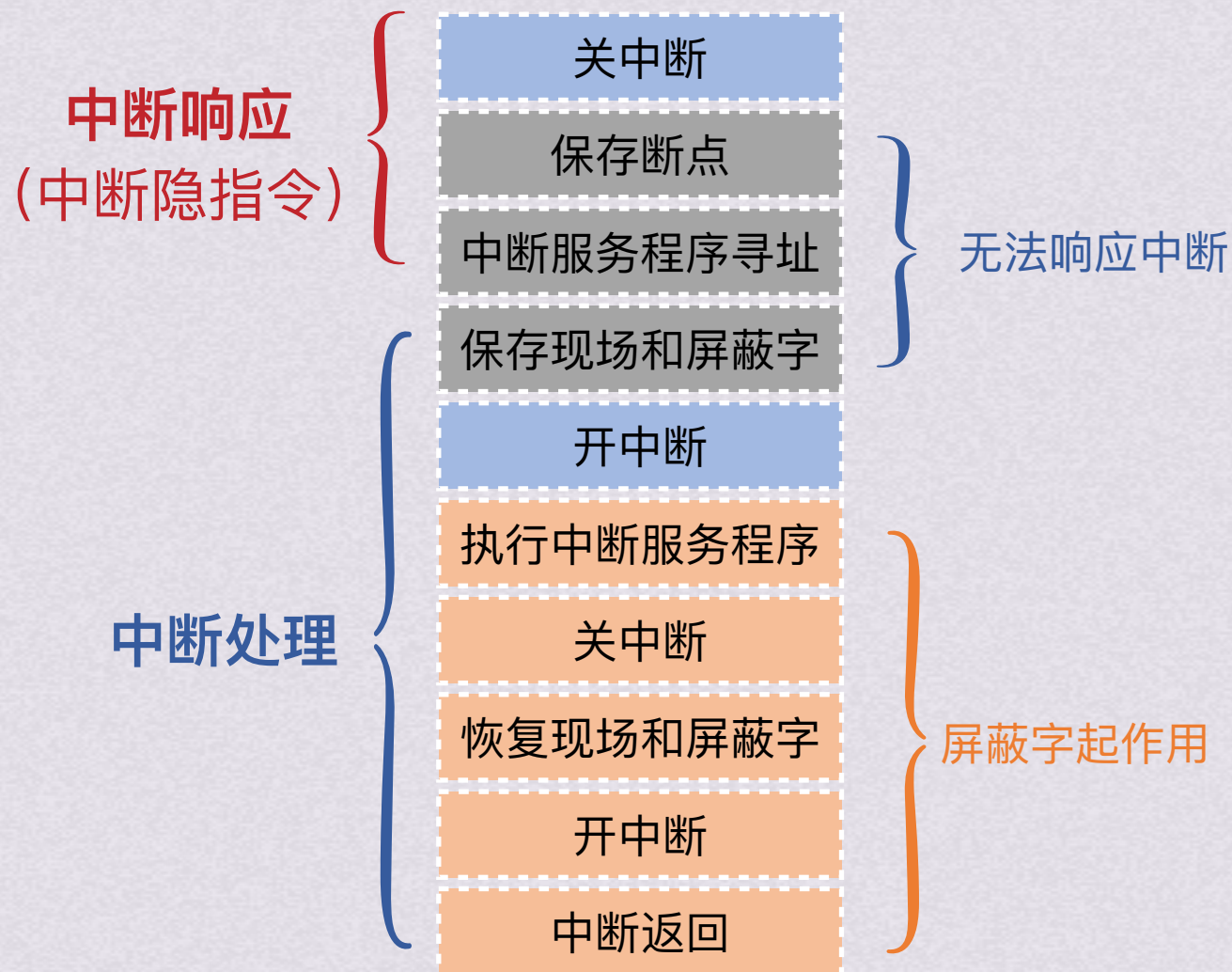




IO数据传送控制方式

中断控制(程序中中断IO方式)

中断处理过程 多重中断



多重中断一般指可屏蔽中断

由于在中断处理过程中才会开中断，因此在隐指令的关中断到开中断之间都不会响应中断

处理阶段开中断后，CPU才会检测多重中断，屏蔽字才开始起作用

只有在第一个中断已经完成中断响应后，才可能被第二个中断给打断



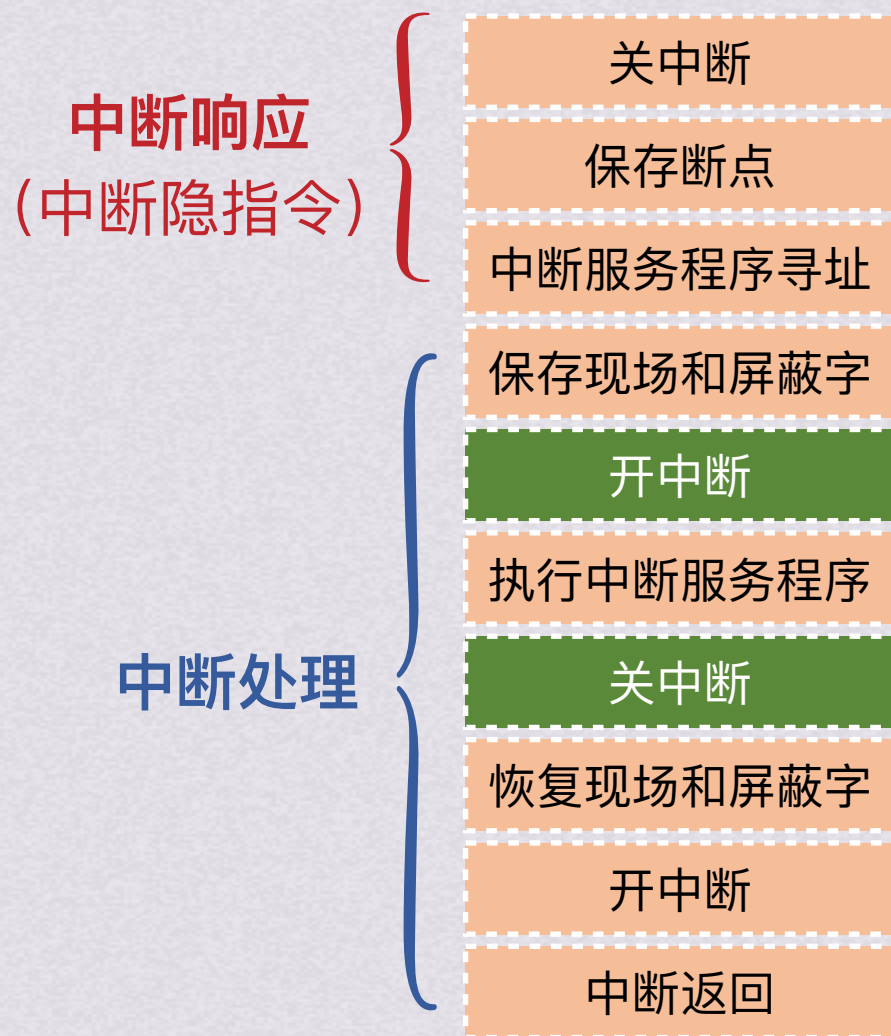
IO数据传送控制方式

中断控制(程序中断IO方式)

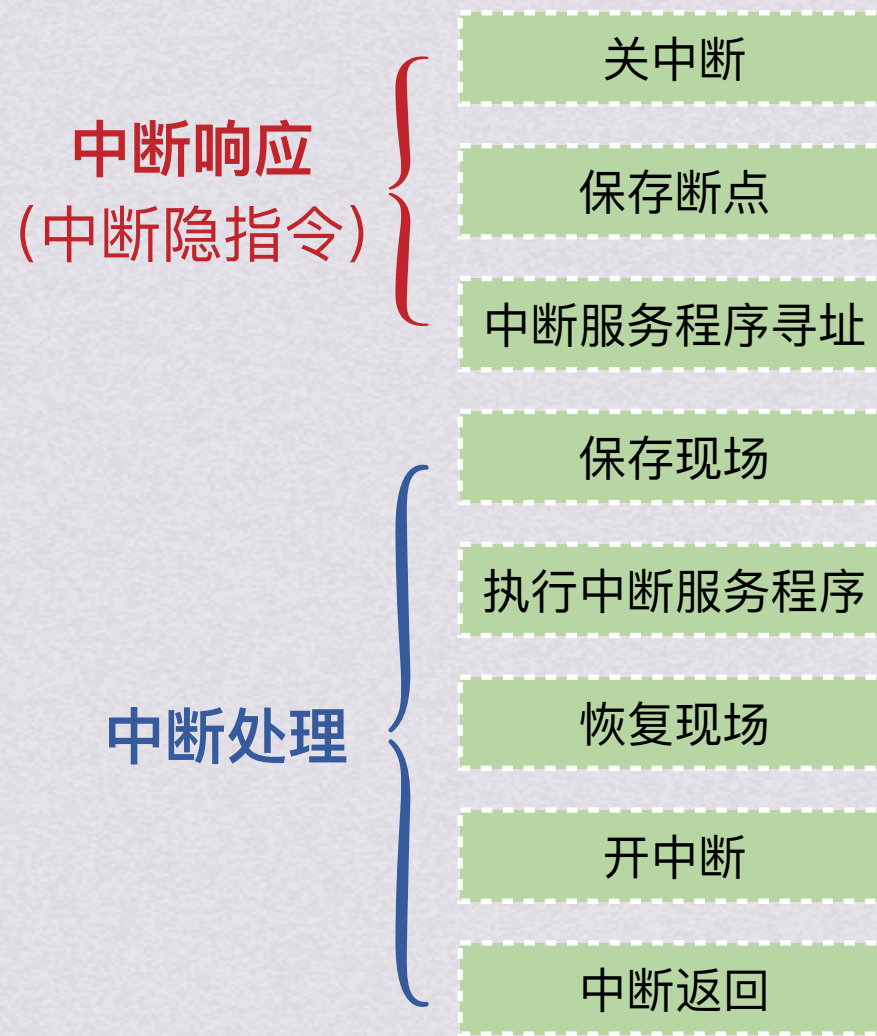
中断处理过程

多重中断

多重中断



单级中断

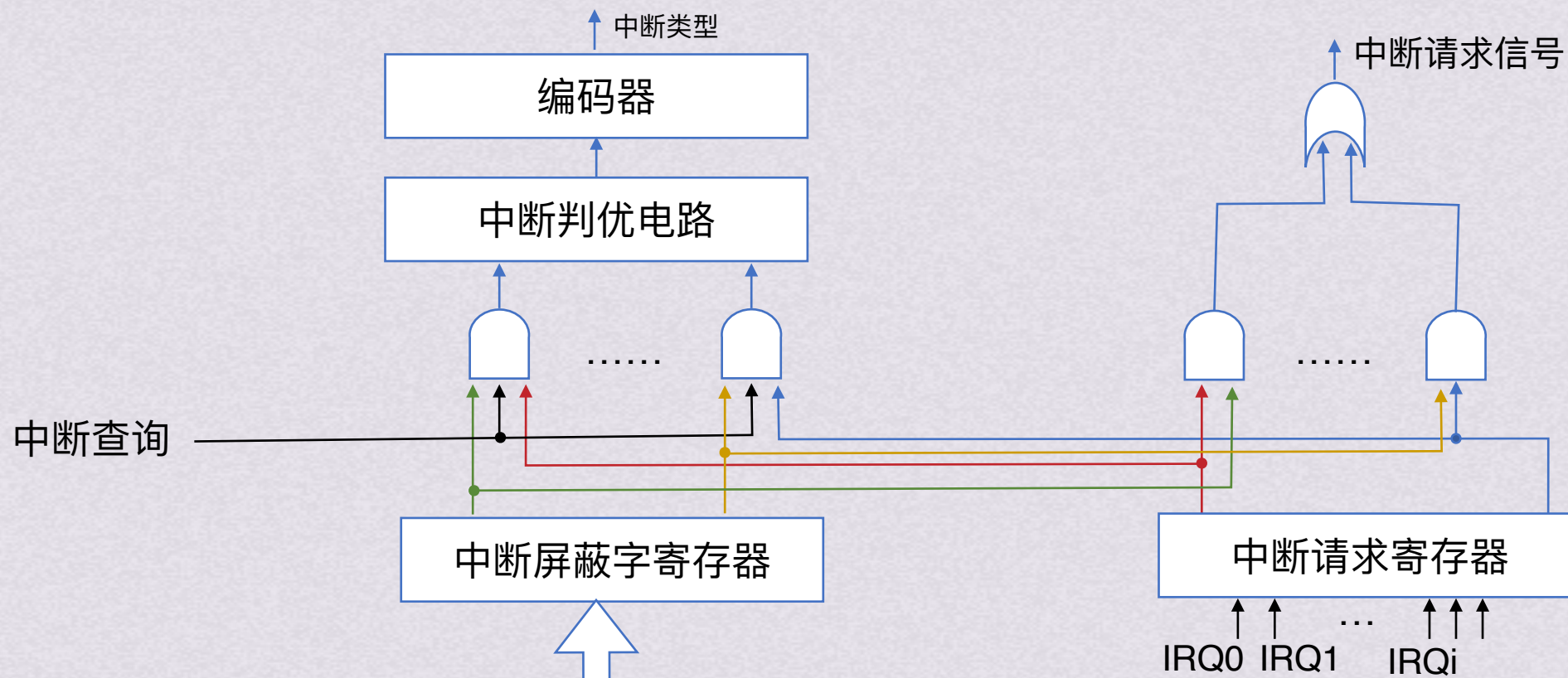




屏蔽字的作用

中断屏蔽

中断系统允许CPU在执行某个中断服务程序时被新的中断请求打断，但有些紧急的中断不允许被新的中断打断，这就是中断屏蔽的概念，通过中断屏蔽字来实现该功能



中断控制器的基本结构



屏蔽字的作用

中断屏蔽

中断系统允许CPU在执行某个中断服务程序时被新的中断请求打断，但有些紧急的中断不允许被新的中断打断，这就是中断屏蔽的概念，通过中断屏蔽字来实现该功能

0表示允许中断

1表示不允许中断(即屏蔽中断)

中断服务程序	屏蔽字			
	1	2	3	4
1	1	1	0	1
2	0	1	0	0
3	1	1	1	1
4	0	1	0	1

处理优先级：3>1>4>2



屏蔽字的作用

中断屏蔽

假定现有4个中断源(A, B, C, D) , 硬件排队次序为A->B->C->D, 各中断源的服务程序中所对应的屏蔽字如下表所示
若4个中断源同时有中断请求, 画出CPU执行程序轨迹

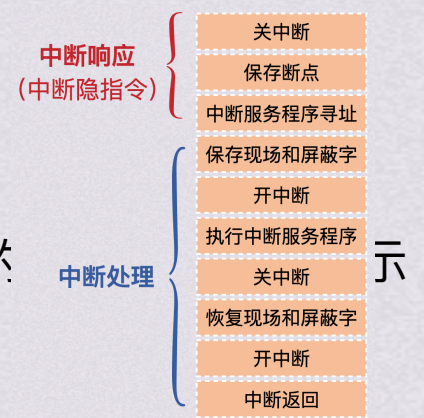
中断服务程序	屏蔽字			
	A	B	C	D
A	1	1	0	1
B	0	1	0	0
C	1	1	1	1
D	0	1	0	1

用0表示允许中断, 1表示不允许中断(即屏蔽中断)

处理优先级: C>A>D>B

屏蔽字的作用

中断屏蔽



假定现有4个中断源(A, B, C, D) , 硬件排队次序为A->B->C->D, 各中断源的服务程序中所对应的

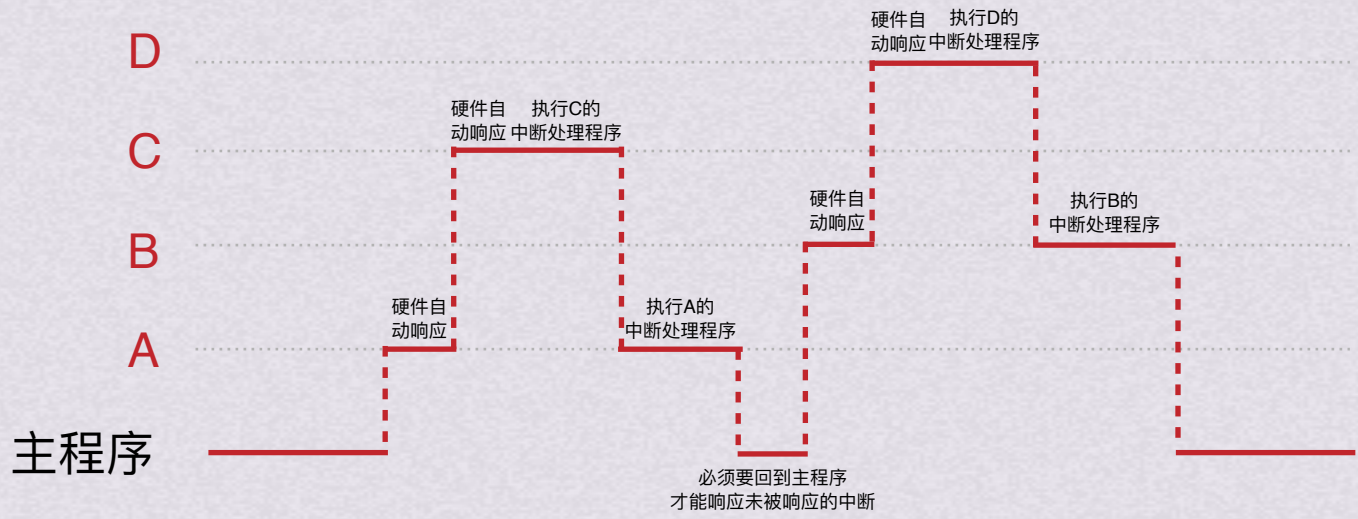
若4个中断源同时有中断请求, 画出CPU执行程序轨迹

中断服务程序	屏蔽字			
	A	B	C	D
A	1	1	0	1
B	0	1	0	0
C	1	1	1	1
D	0	1	0	1

用0表示允许中断, 1表示不允许中断(即屏蔽中断)

处理优先级: C>A>D>B

- 1.先通过响应优先级确定响应顺序
- 2.响应结束后需要参考处理优先级确定处理顺序

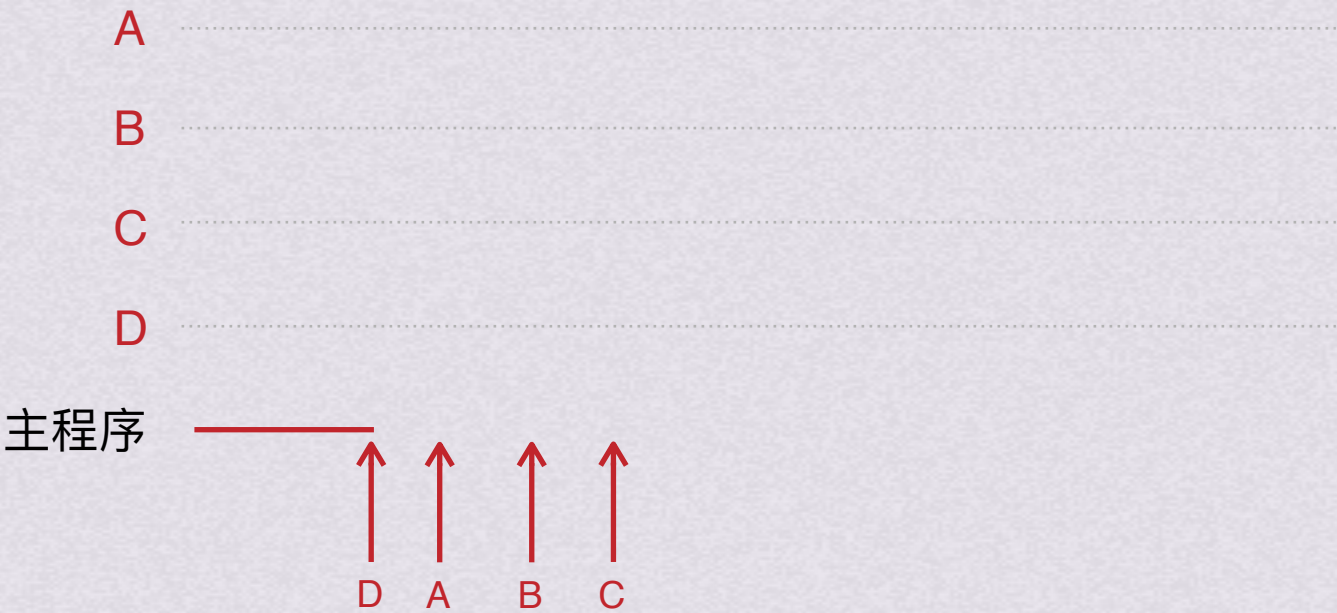


屏蔽字的作用

中断屏蔽

设某计算机有4级中断ABCD，其硬件排队优先级次序为A>B>C>D，如表所示列出了执行每级中断服务程序所需时间
中断优先级C>A>D>B；中断请求在图中标出，他们的中断屏蔽码如表所示，请画出CPU执行中断服务程序的序列

中断源	中断屏蔽码			
	A	B	C	D
A	1	1	0	1
B	0	1	0	0
C	1	1	1	1
D	0	1	0	1

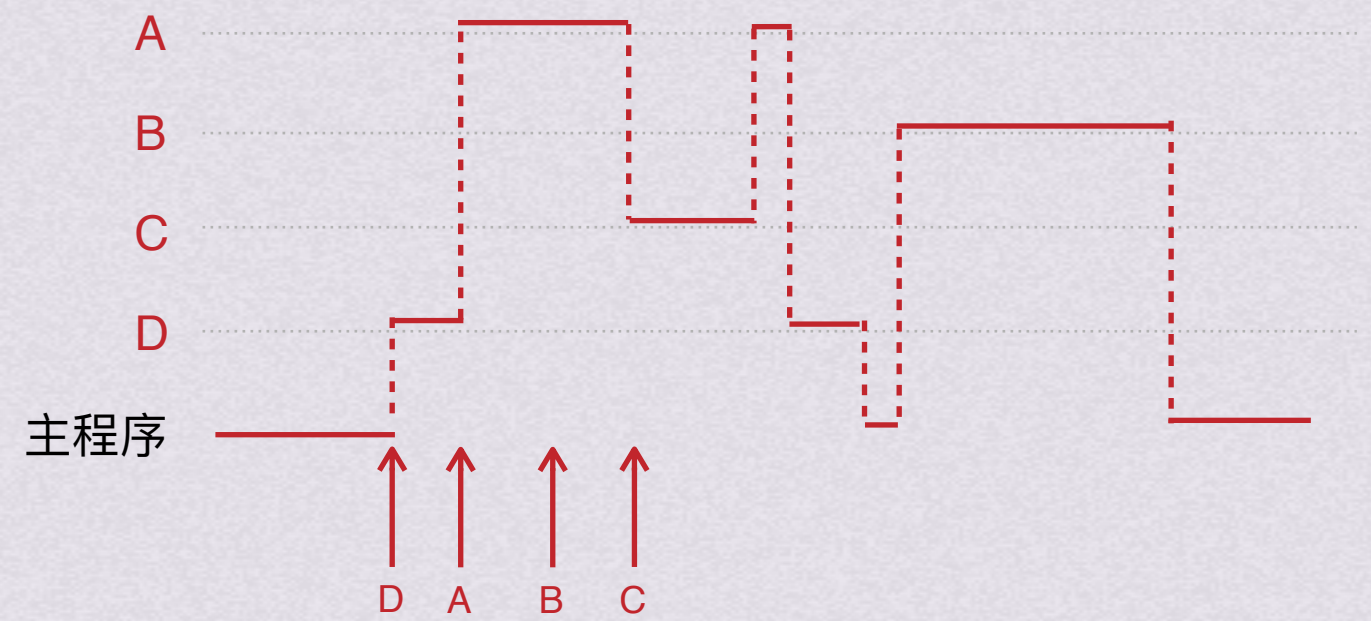


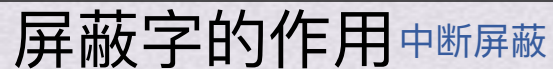
屏蔽字的作用

中断屏蔽

设某计算机有4级中断ABCD，其硬件排队优先级次序为A>B>C>D，如表所示列出了执行每级中断服务程序所需时间
中断优先级C>A>D>B；中断请求在图中标出，他们的中断屏蔽码如表所示，请画出CPU执行中断服务程序的序列

中断源	中断屏蔽码			
	A	B	C	D
A	1	1	0	1
B	0	1	0	0
C	1	1	1	1
D	0	1	0	1





中断源	中断屏蔽码			
	A	B	C	D
A	1	1	0	1
B	0	1	0	0
C	1	1	1	1
D	0	1	0	1



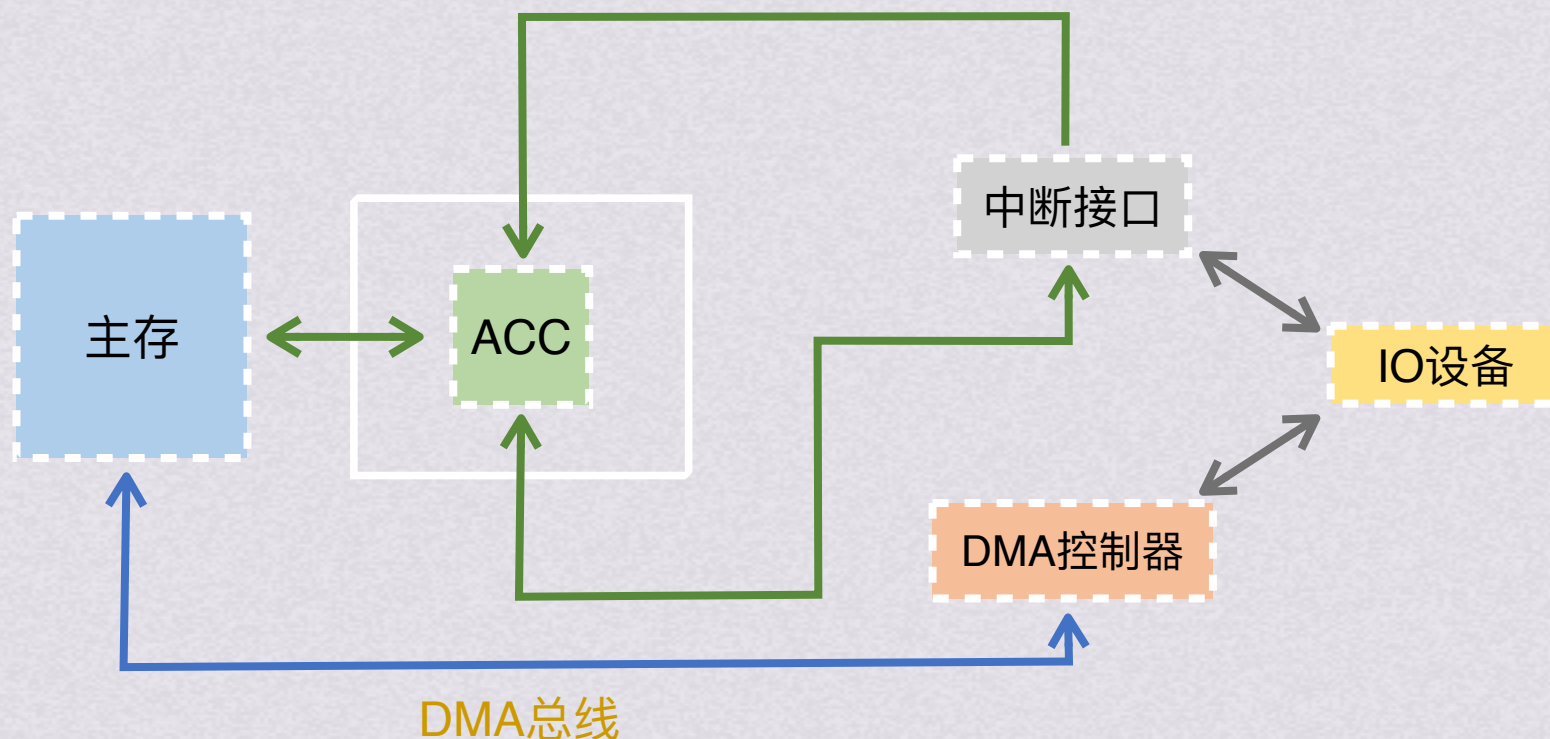


DMA控制

DMA (Direct Memory Access)即直接存储器存取，用专门的DMA接口硬件控制外设、主存间直接数据交换（不经过CPU）

通常把专门用来控制总线进行DMA传送的接口硬件称为DMA控制器

DMA传送时，CPU让出总线控制权，由DMA控制器控制总线，通过“挪用”一个主存周期完成和主存之间的一次数据交换，或独占若干个主存周期完成一批数据的交换





DMA

主要用于磁盘等高速设备数据传送

采用“请求-响应”方式



主要用于磁盘等高速设备数据传送 这类设备大多采用成批数据交换方式

采用“请求-响应”方式 中断IO方式请求的是处理器的时间，DMA方式请求的是总线控制权



三种DMA方式

CPU停止法

DMA传输时，由DMA控制器发一个停止信号给CPU，使CPU脱离总线，停止访问主存，直到DMA传送一块数据结束

周期挪用法

DMA传输时，CPU让出一个总线事务周期，由DMA控制器挪用一個主存周期来访问主存，传送完一个数据后立即释放总线，是一种单字传输方式

交替分时访问法

每个存储周期分成两个时间片，一个给CPU，另一个给DMA控制器，这样在每个CPU周期内，CPU和控制器都可访问存储器

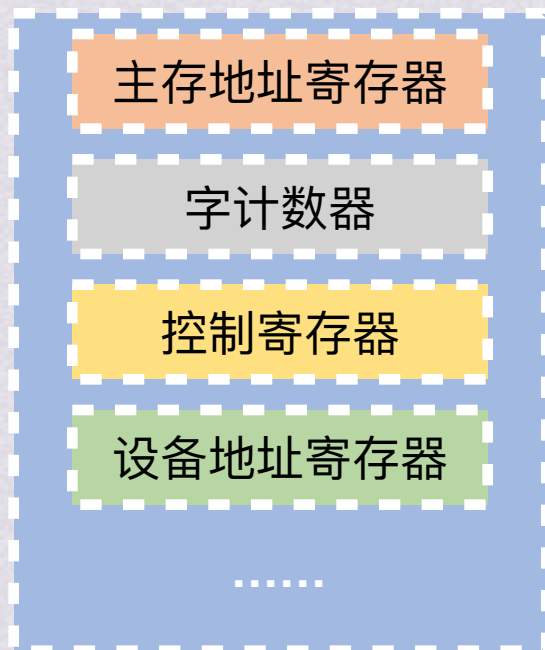
因DMA控制器和CPU共享主存，所以可能出现二者争用主存的现象，通过这三种方法来让他们协调使用内存



DMA操作步骤



DMA操作步骤



DMA控制器

DMA控制器内有各种寄存器用于存放IO传送所需的各种参数
并且还有其他控制逻辑，能控制设备通过总线与主存直接交换数据



DMA操作步骤

第一步：DMA控制器的初始化

(1)准备内存区：若是从外设输入数据，则进行内存缓冲区的申请，对缓冲区初始化；若是输出数据到外设，则在内存准备好数据

(2)设置传送参数：执行输入输出指令来测试外设状态，并对DMA控制器设置各种参数





DMA操作步骤

第一步：DMA控制器的初始化

(1)准备内存区：若是从外设输入数据，则进行内存缓冲区的申请，对缓冲区初始化；若是输出数据到外设，则在内存准备好数据

(2)设置传送参数：执行输入输出指令来测试外设状态，并对DMA控制器设置各种参数

(3)发送“启动DMA传送”命令，然后调度CPU执行其他进程

第二步：DMA数据传送

CPU预置好DMA传送参数并发送“启动DMA传送”命令后，就把数据传送的工作交给了DMA控制器，整个DMA传送过程中不再需要CPU参与

DMA控制器给出内存地址，读写线发命令，数据线给数据，每完成一个数据传送，字计数值减1并修改主存地址，字计数器位0时完成所有IO操作

第三步：DMA结束处理（后处理）

字计数器=0，则发出“DMA结束”中断请求信号给CPU，转入中断服务程序，做一些数据校验等后处理工作



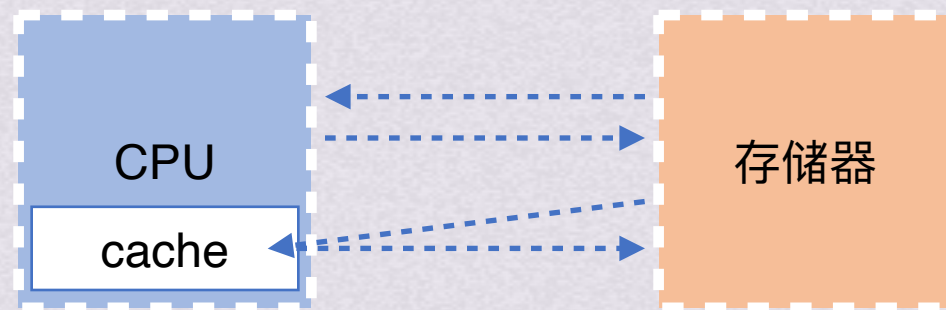


DMA与存储系统



DMA与存储系统

没有DMA控制器时，所有对存储器的访问都来自CPU，通过MMU中的地址转换和cache访问来进行存储器存取

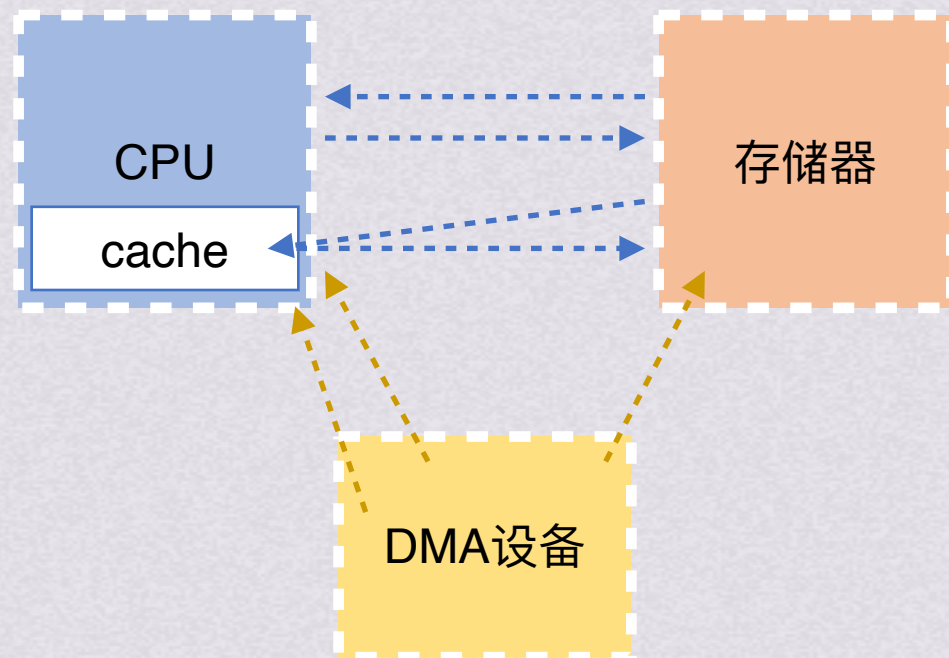




DMA与存储系统

没有DMA控制器时，所有对存储器的访问都来自CPU，通过MMU中的地址转换和cache访问来进行存储器存取

有了DMA后，系统中就有了另一个访问存储器的路径，它不通过地址转换机制和cache层次。这样，在虚拟存储器和cache系统中就会产生一些问题。这些问题的解决通常要结合硬件和软件两方面的技术支持

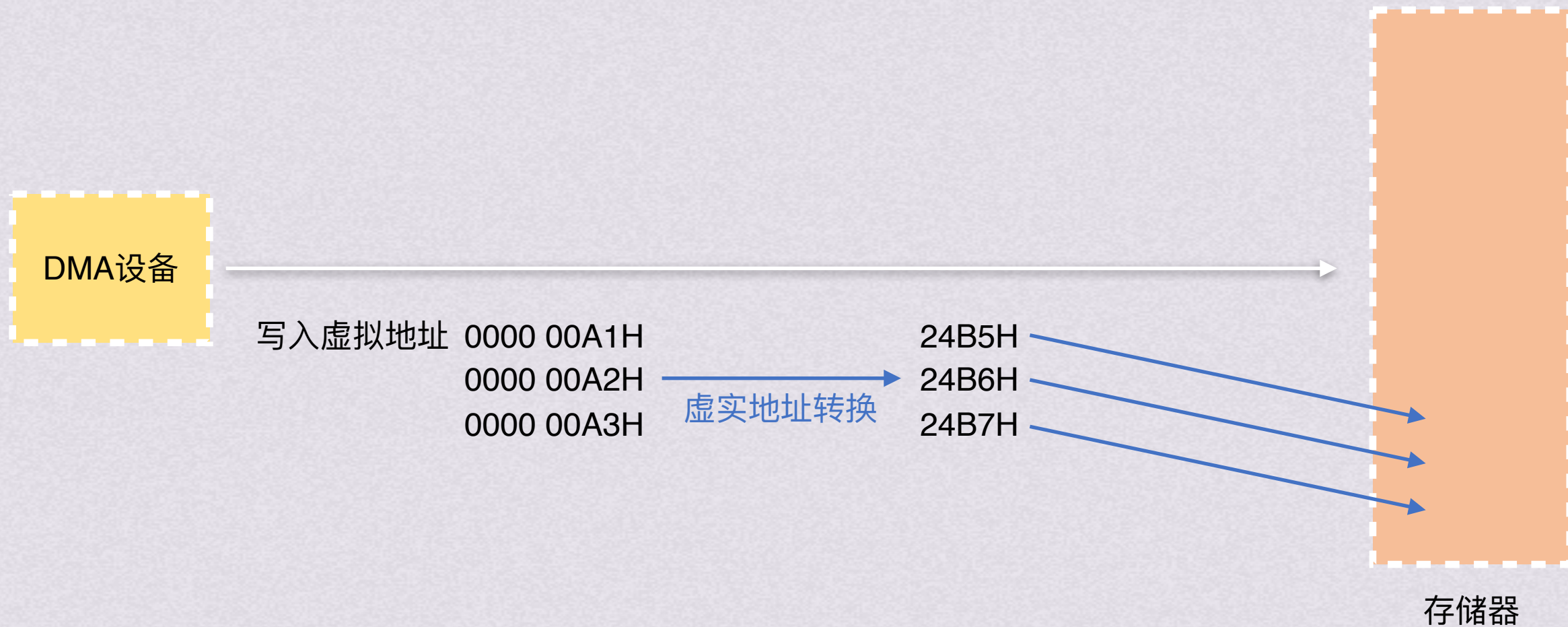




DMA与存储系统

DMA以虚拟地址还是以物理地址工作？

如果使用虚拟地址，就在DMA接口中就必须将虚实地址转换





DMA与存储系统

DMA以虚拟地址还是以物理地址工作？

如果使用虚拟地址，就在DMA接口中就必须将虚实地址转换

如果使用物理地址，每次DMA传送不能跨页





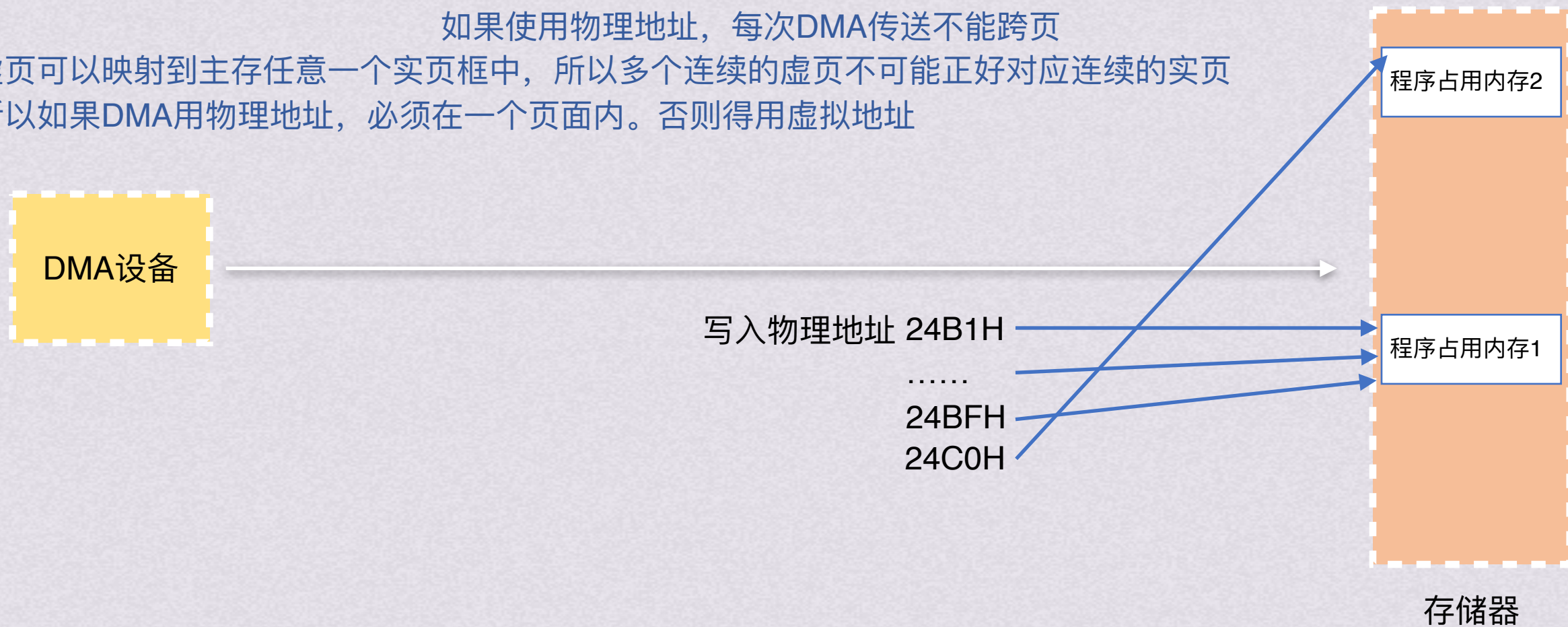
DMA与存储系统

DMA以虚拟地址还是以物理地址工作？

如果使用虚拟地址，就在DMA接口中就必须将虚实地址转换

如果使用物理地址，每次DMA传送不能跨页

每个虚页可以映射到主存任意一个实页框中，所以多个连续的虚页不可能正好对应连续的实页框。所以如果DMA用物理地址，必须在一个页面内。否则得用虚拟地址





DMA与存储系统

DMA以虚拟地址还是以物理地址工作？

如果使用物理地址，每次DMA传送不能跨页
每个虚页可以映射到主存任意一个实页框中，所以多个连续的虚页
不可能正好对应连续的实页框。所以如果DMA用物理地址，必须在一个页面内。否则得用虚拟地址

如果使用虚拟地址，就在DMA接口中就必须将虚实地址转换

使用虚拟地址的情况，DMA接口中应该有一个类似页表的地址映射表，用于虚实地址转换





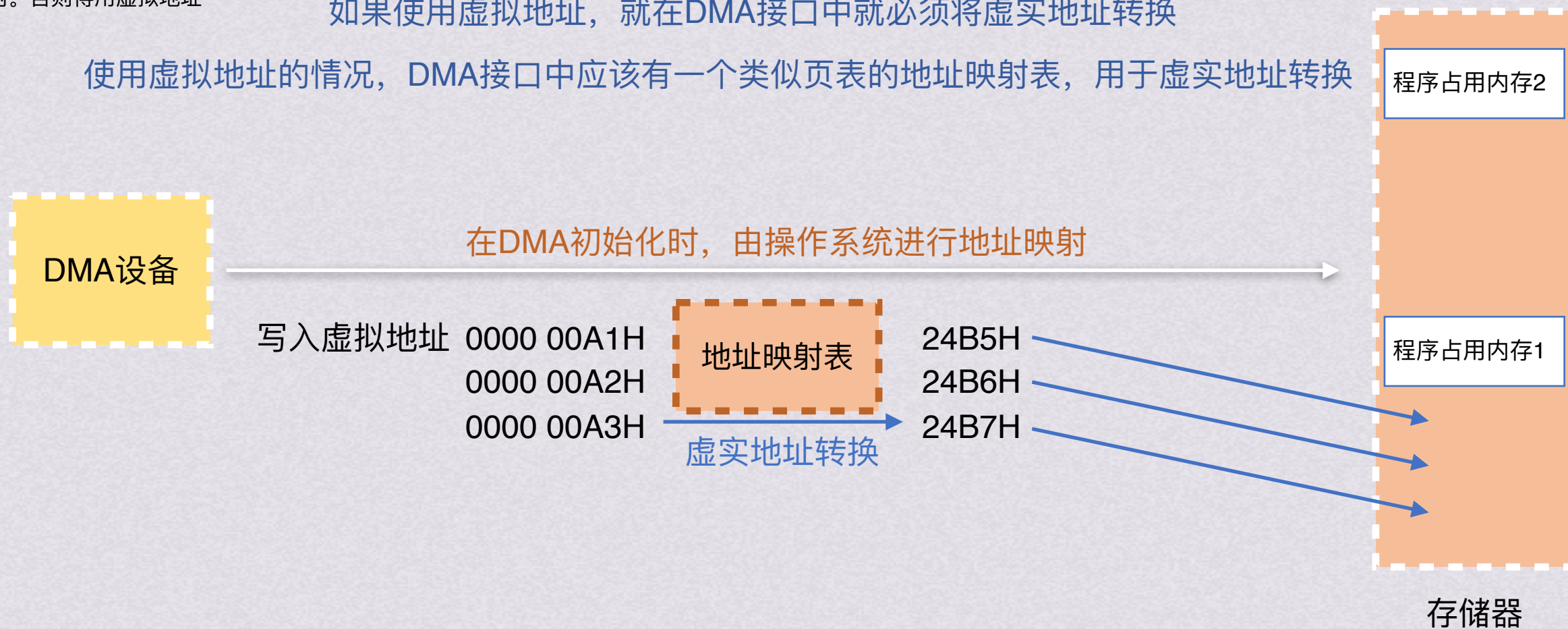
DMA与存储系统

DMA以虚拟地址还是以物理地址工作？

如果使用物理地址，每次DMA传送不能跨页
每个虚页可以映射到主存任意一个实页框中，所以多个连续的虚页
不可能正好对应连续的实页框。所以如果DMA用物理地址，必须在一个页面内。否则得用虚拟地址

如果使用虚拟地址，就在DMA接口中就必须将虚实地址转换

使用虚拟地址的情况，DMA接口中应该有一个类似页表的地址映射表，用于虚实地址转换

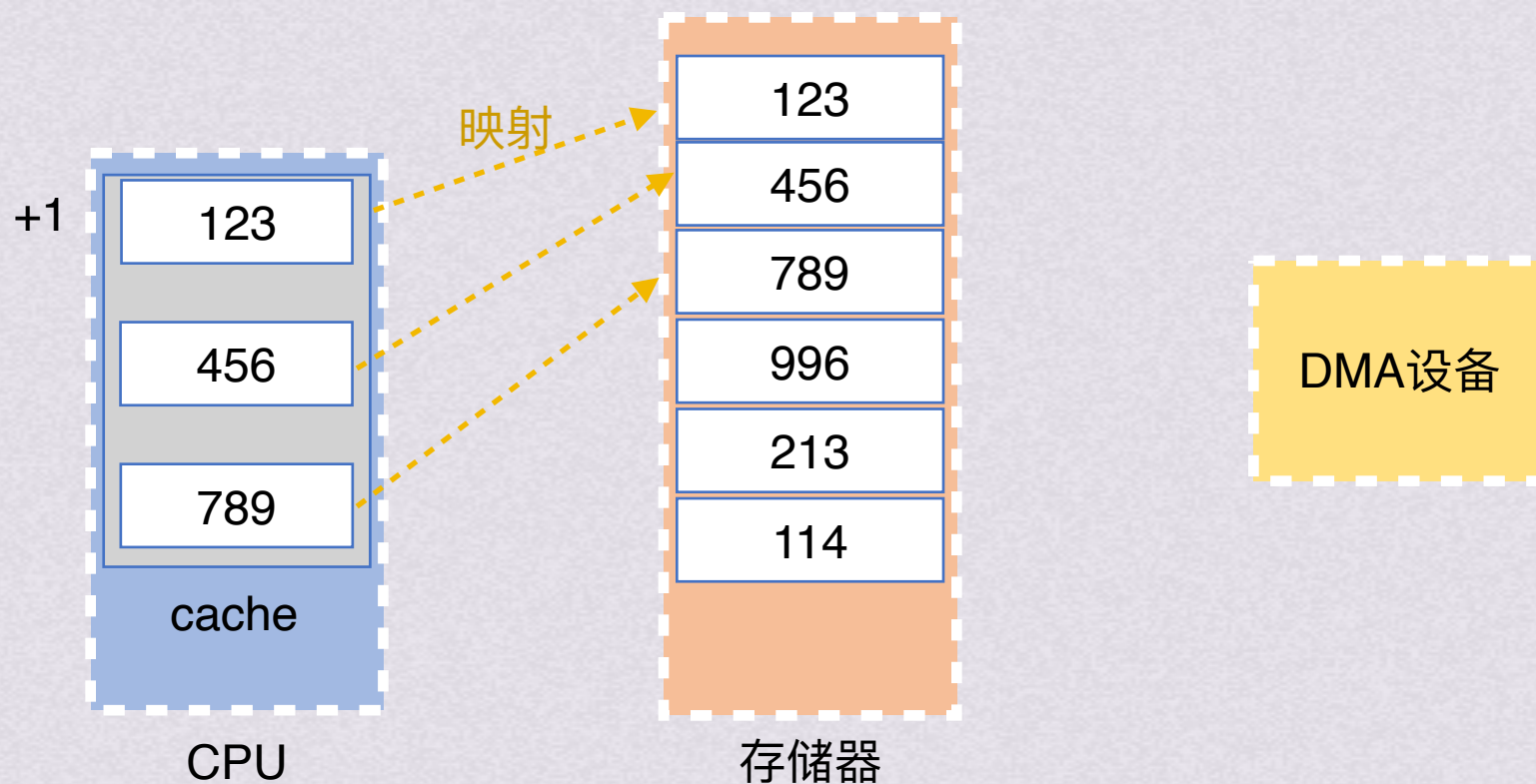




DMA与存储系统

DMA&Cache

具有DMA的cache系统也会产生问题。DMA控制器直接向存储器发出访存请求而不通过cache，DMA看到的主存单元的值和CPU看到的cache中的副本可能不同

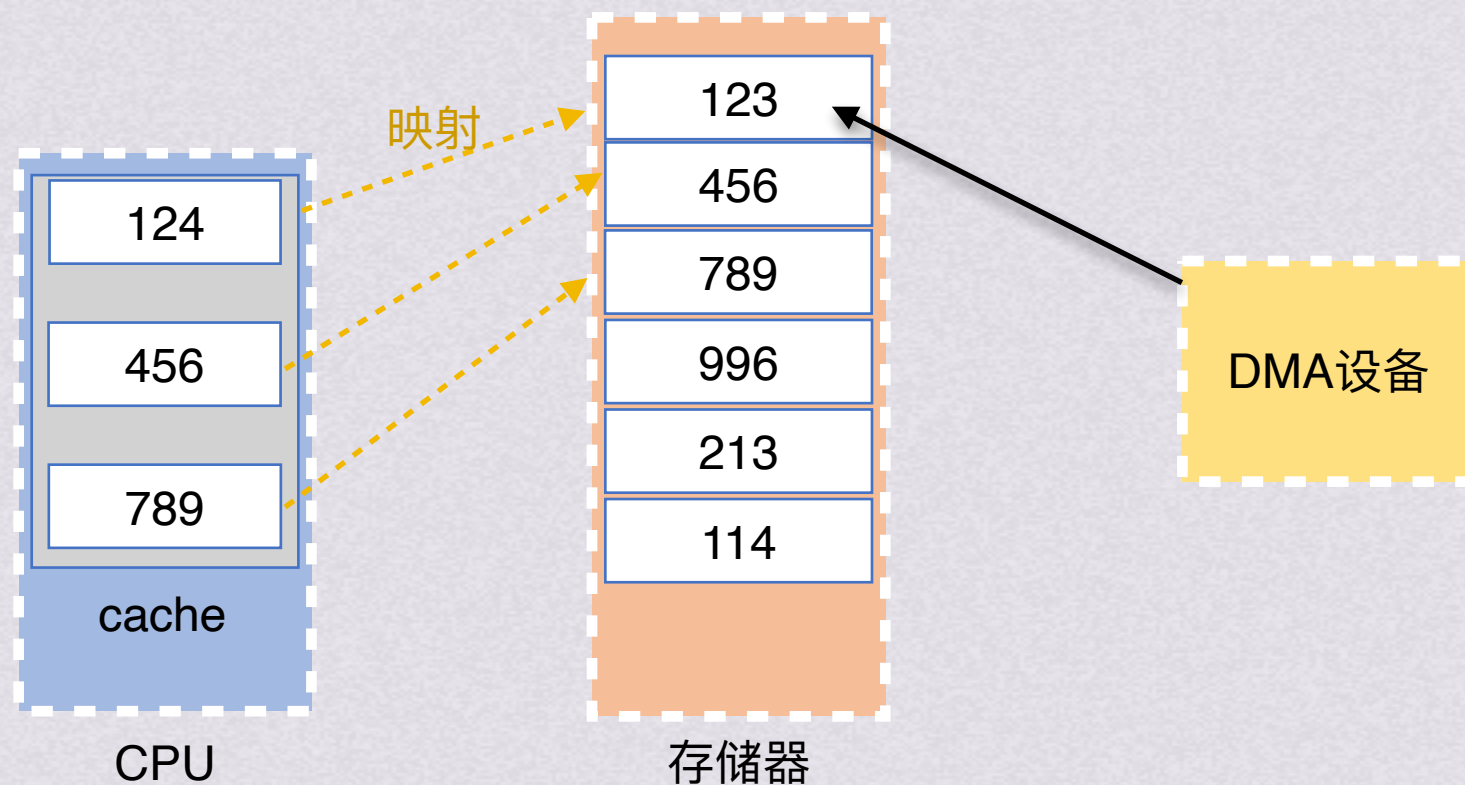




DMA与存储系统

DMA&Cache

具有DMA的cache系统也会产生问题。DMA控制器直接向存储器发出访存请求而不通过cache，DMA看到的主存单元的值和CPU看到的cache中的副本可能不同

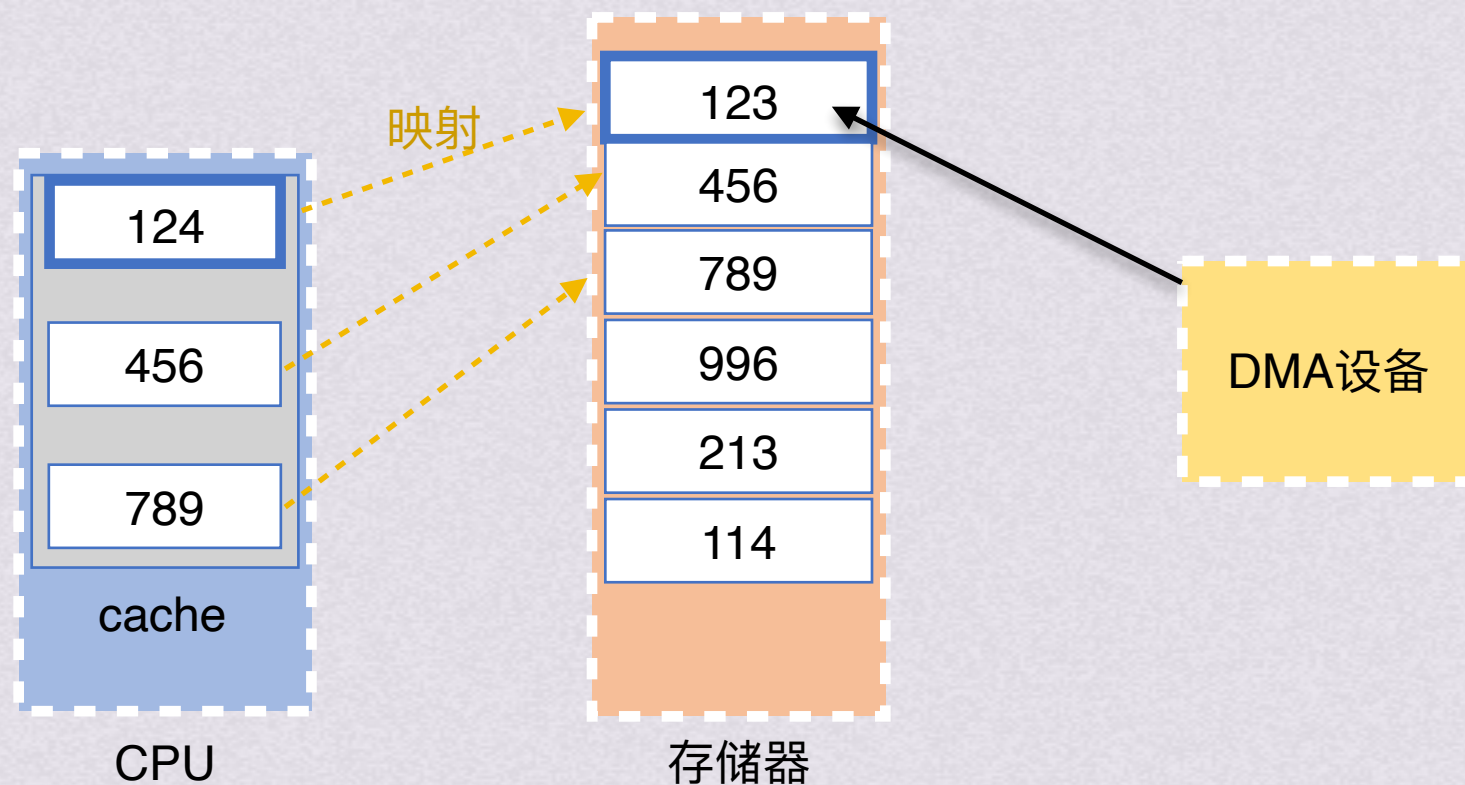




DMA与存储系统

DMA&Cache

具有DMA的cache系统也会产生问题。DMA控制器直接向存储器发出访存请求而不通过cache，DMA看到的主存单元的值和CPU看到的cache中的副本可能不同





DMA与存储系统

DMA&Cache

具有DMA的cache系统也会产生问题。DMA控制器直接向存储器发出访存请求而不通过cache，DMA看到的主存单元的值和CPU看到的cache中的副本可能不同

解决方法：

- 1.让IO活动通过cache进行
- 2.让操作系统在IO读时有选择的使某些cache块无效，在IO写时迫使cache进行一次写回操作（这种操作常被称cache刷新）需要少量硬件支持
- 3.通过一个硬件机制来选择被刷新或使无效的cache项，这种用硬件方式来保证cache一致性的方式大多被用在多处理器系统中

